스프링과 함께하는 레퍼런스 기술모음

사수가 던진 티켓 아홉 장, 3개월의 기록

Docker부터 RabbitMQ까지, 실무에서 마주치는 9가지 기술을 하나의 프로젝트로 경험합니다. 사수가 건넨 티켓을 하나씩 해결하며 Spring Boot 생태계의 레퍼런스 기술을 익힙니다.

RAG 실전 가이드

목차

서문 4

프롤로그 6

1장. Docker로 시작하는 개발 환경 8
- 입사 첫날의 한마디 8
- 이것만은 기억하자 15

2장. 카카오 로그인이 두려웠던 이유 16
- 카카오 로그인 붙여주세요 16
- 이것만은 기억하자 27

3장. 서버가 늘었는데 로그인이 풀립니다 28
- 서버 2대의 배신 28
- 이것만은 기억하자 35

4장. 프로필 사진 좀 넣어주세요 36
- 사진 한 장이 이렇게 어려울 줄은 36
- 이것만은 기억하자 43

5장. 서버가 다 할 필요는 없잖아요 44
- 디스크 사용량 90% 44
- 이것만은 기억하자 56

6장. 새 댓글이 달렸는데 왜 새로고침을 해야 하죠 57
- 알림이 안 와요 57
- 이것만은 기억하자 65

7장. 영상도 WebSocket으로 되지 않아? 67
- 1GB짜리 영상 67
- 이것만은 기억하자 74

8장. 검색이 왜 이렇게 느려요? 75

5초짜리 검색 75
기술 파트 76
이것만은 기억하자 84

9장. 그 알림, 꼭 직접 전해야 해? 85
수동 동기화의 한계 85
기술 파트 86
이것만은 기억하자 95

맺음말 97

서문

Spring Boot로 REST API를 만들 줄 아는 분이라면, 채용 공고를 한번 열어 보신 적이 있을 겁니다. Docker, Redis, Elasticsearch, Kafka, WebSocket. 이름은 익숙한데 직접 써본 적은 없는 기술들이 자격 요건 칸에 줄지어 있습니다.

CRUD는 만들 수 있는데 그 다음이 막막합니다.

이 책에서 다루는 아홉 가지 기술은 전부 그 "그 다음"에 해당합니다. 한 가지씩 왜 필요한지부터 짚어 보겠습니다.

서버 한 대에서 개발하던 시절에는 로컬에 JDK 깔고 IDE에서 Run 버튼을 누르면 끝이었습니다. 그런데 팀원이 늘고 환경이 달라지는 순간, "내 컴퓨터에서는 되는데요"라는 말이 시작됩니다. Docker는 이 문제를 해결하기 위해 등장했고, 지금은 대부분의 팀에서 개발 환경을 컨테이너로 맞추고 있습니다. 선택이 아니라 기본 인프라가 된 지 오래입니다.

로그인도 마찬가지입니다. 직접 아이디와 비밀번호를 받아서 처리하는 서비스는 점점 줄고 있습니다. 카카오, 구글, 네이버 같은 소셜 로그인을 기대하는 사용자가 대부분이고, 그 뒤에는 OAuth 2.0이라는 표준이 있습니다. 시퀀스 다이어그램 화살표가 많아서 처음엔 복잡해 보이지만, 결국 토큰을 주고받는 하나의 흐름입니다.

서버가 한 대일 때는 세션을 메모리에 두면 됩니다. 그런데 트래픽이 늘어서 서버를 두 대로 늘리면, 한 쪽에서 로그인한 사용자가 다른 쪽으로 넘어갈 때 로그인이 풀립니다. Redis는 이런 상태 공유 문제를 해결하는 가장 흔한 선택지입니다. 세션뿐 아니라 캐시, 랭킹, 실시간 데이터까지 쓰임이 넓어서 사실상 표준 인메모리 저장소로 자리 잡았습니다.

파일 업로드는 언제나 처음 접하면 낯선 영역입니다. 이미지를 서버로 어떻게 보내는지, 저장은 어디에 하는지, 용량이 커지면 어떻게 하는지. Base64 인코딩으로 시작해서 S3와 Lambda로 넘어가는 과정은 서버가 모든 걸 직접 처리하지 않아도 된다는 사실을 알려 줍니다. 클라우드와 서버리스는 특별한 기술이 아니라, 서버 디스크가 가득 찼을 때 자연스럽게 찾게 되는 해결책입니다.

사용자는 새로고침을 누르지 않아도 알림이 바로 뜨길 기대합니다. 실시간 통신이라는 말이 거창하게 들리지만, 방법은 크게 세 가지입니다. 폴링, SSE, 웹소켓. 어떤 상황에 뭘 써야 하는지 직접 비교해 보면 선택 기준이 명확해집니다.

영상 콘텐츠를 다루는 서비스는 계속 늘고 있습니다. 교육 플랫폼, 사내 영상 공유, 라이브 커머스. 1GB짜리 파일을 통째로 내려받게 할 수는 없으니, 잘게 쪼개서 순서대로 보내는 HLS 스트리밍 방식을 알아둘 필요가 있습니다.

데이터가 쌓이면 검색이 느려집니다. SQL의 LIKE 검색은 데이터가 많아질수록 한계가 뚜렷합니다. Elasticsearch는 역인덱스라는 구조로 이 문제를 해결하는데, 검색 품질이 곧 서비스 경쟁력이 되는 시대에 알아두면 쓸모가 많습니다.

마지막으로 시스템이 커지면 서비스끼리 직접 호출하는 구조가 부담이 됩니다. 한쪽이 느려지면 다른 쪽까지 느려지고, 한쪽이 죽으면 연쇄로 장애가 납니다. RabbitMQ 같은 메시지 큐는 이 결합을 끊어주는 역할을 합니다. 마이크로서비스 아키텍처로 전환하는 팀이 늘면서, 메시지 큐는 거의 필수 인프라가 되었습니다.

이 아홉 가지 기술에는 공통점이 하나 있습니다. 전부 “들어는 봤는데 직접 해본 적은 없는” 것들이라는 점입니다.

이 책은 깊이 파고드는 레퍼런스가 아닙니다. 처음 접하는 기술 앞에서 전체 그림을 잡고, 로컬에서 한 번 돌려 보고, 두려움을 걷어내는 것이 목표입니다. 각 장은 독립적이니 관심 있는 기술부터 골라 읽어도 됩니다.

Spring Boot로 CRUD를 만들어본 경험이 있다면 준비는 끝났습니다.

프롤로그

입사 첫 주, 사수 가 모니터 옆에 포스트잇 한 장을 붙여 주었습니다.

사수: “일단 이거부터 해봐. 로컬에 환경 띄우는 거야.”

Docker 라는 단어를 그때 처음 봤습니다. 내 컴퓨터에서 왜 안 돌리고 컨테이너라는 걸 쓰는 건지, 이해보다 설치가 먼저였습니다. 터미널에 명령어를 치고 빌드가 끝났을 때, 그게 이 프로젝트에서 넘긴 첫 번째 관문이었습니다.

일주일 뒤, 사수 가 슬쩍 JIRA 티켓 하나를 옮겨 놓았습니다. 카카오 로그인. OAuth 2.0 시퀀스 다이어그램의 화살표가 열다섯 개쯤 되었는데, 세 번을 읽어도 누가 누구에게 뭘 보내는지 감이 오지 않았습니다. (이걸 나보고 하라고?)

간신히 로그인을 붙이고 나니, 서버를 한 대 더 띄우자는 이야기가 나왔습니다. 그런데 서버가 두 대가 되자 로그인이 풀리기 시작했습니다. 사수 가 한마디 했습니다.

사수: “세션 어디 저장하고 있어?”

Redis 라는 이름을 그때 처음 들었습니다. 세션이 한쪽 서버에만 남아 있었다는 걸 알기까지 하루가 걸렸습니다.

그 다음은 프로필 사진이었습니다. 이미지를 어떻게 서버로 보내는지 고민하다가 Base64 인코딩을 알게 되었고, 그 다음 주에는 서버 디스크 사용량 90퍼센트 알림이 울렸습니다. 파일을 서버 밖으로 빼야 한다는 말에 S3 와 Lambda 를 처음 만났습니다. 사수는 매번 같은 패턴이었습니다. 티켓을 넘기고, 막히면 힌트를 주고, 나머지는 알아서 부딪히게 두었습니다.

사수: “이번엔 알림이야. 새로고침 안 누르고 바로 떠야 해.”

실시간이라는 말이 그렇게 무거운 줄 몰랐습니다. 폴링, SSE, 웹소켓 세 가지 중 뭘 써야 하는지, 아무리 검색해도 명쾌한 답이 없었습니다. 결국 셋 다 짜보고 나서야 감이 잡혔습니다.

그러고 나서 1GB짜리 교육 영상을 올렸더니 브라우저가 멈췄습니다. 영상을 잘게 쪼개서 보내는 HLS 라는 방식을 그때야 알았습니다. 두 달 반쯤 지났을 때는 검색이 5초나 걸린다는 버그 리포트가 올라왔습니다. LIKE 검색으로 버티던 구조를 Elasticsearch 로 바꾸면서, 검색이 단순히 문자열을 찾는 일이 아니라는 걸 깨달았습니다.

마지막으로 넘겨받은 티켓은 시스템 간 데이터 동기화였습니다. 수동으로 맞추다 보니 빠지는 건이 생기고 순서가 꼬였습니다. RabbitMQ 를 도입하면서, 메시지를 큐에 넣고 순서대로 처리하는 구조가 사람 손을 얼마나 덜어 주는지 알게 되었습니다.

돌아보면 3개월, 티켓 아홉 장이었습니다.

매번 모르는 단어 앞에서 멈췄고, 검색하고, 틀리고, 다시 읽었습니다. 대단한 깨달음 같은 건 없었습니다. 사수가 던져 준 티켓을 하나씩 넘겼을 뿐입니다.

이 책은 그 아홉 장의 티켓 기록입니다. 지금 첫 번째 티켓 앞에서 커서만 바라보고 있는 분이 있다면, 한 가지만 말씀드리겠습니다. 저도 거기서 시작했습니다.

1장. Docker로 시작하는 개발 환경

입사 첫날의 한마디

출근 첫날, 모니터 앞에 앉자마자 팀장 이 다가왔습니다.

팀장: “환경 세팅은 Docker로 하면 돼요. README에 있으니까 보고 따라 하세요.”

고개를 끄덕였지만 손이 움직이지 않았습니다. Docker. 이름은 수십 번 들었습니다. 면접 준비할 때 “컨테이너 기반 가상화 플랫폼”이라고 외운 적도 있습니다. 하지만 직접 써본 적은 한 번도 없었습니다.

(일단 자바부터 깔자. MySQL도 깔고. 그건 해봤으니까.)

익숙한 방식으로 시작했습니다. OpenJDK를 내려받고, MySQL 설치 파일을 찾고, 환경 변수를 잡았습니다. 한 시간쯤 지났을까. 빌드 버튼을 눌렀는데 에러가 터졌습니다. 자바 버전이 맞지 않았습니다. 프로젝트는 21을 쓰고 있었고, 제 컴퓨터에는 17이 깔려 있었습니다.

버전을 바꾸고 다시 빌드. 이번에는 MySQL 포트가 충돌했습니다. 예전에 깔아둔 MariaDB가 3306을 쓰고 있었습니다.

옆자리 선배 가 슬쩍 모니터를 보더니 한마디 했습니다.

선배: “그래서 Docker 쓰라니까. 나도 처음에 그랬어.”

이사를 떠올려 보겠습니다.

새 집에 이사하면 냉장고, 세탁기, 가스레인지를 하나씩 설치합니다. 전압이 맞는지 확인하고, 수도 배관을 연결하고, 가스 밸브를 열어야 합니다. 집마다 배관 위치가 다르고, 콘센트 규격이 다르고, 가스 종류가 다릅니다. 같은 냉장고인데 이 집에서는 되고 저 집에서는 안 되는 일이 생깁니다.

그런데 만약 냉장고가 자기만의 전기, 자기만의 배관, 자기만의 공간을 통째로 가지고 다닌다면 어떨까요. 어느 집에 가져다 놓든 플러그 하나만 꽂으면 바로 돌아가는 겁니다. 집 구조를 신경 쓸 필요가 없습니다. Docker 가 하는 일이 그것입니다. 애플리케이션이 돌아가는 데 필요한 모든 것을 하나의 상자에 담습니다. 자바 버전, 라이브러리, 설정 파일까지 전부요. 그 상자를 어떤 컴퓨터에 올려놓든 똑같이 동작합니다.

팀장 이 “Docker로 환경 맞추세요”라고 말한 건, “이 상자를 열어서 실행만 하세요”라는 뜻이었습니다. 제가 자바를 직접 깔고, MySQL을 직접 잡고, 포트 충돌을 직접 해결한 건 — 빈집에 배관부터 새로 놓은 셈이었습니다. 선배 가 docker-compose 명령어 하나를 알려줬습니다. 터미널에 입력하자 서버가 올라왔습니다. 자바 설치도, MySQL 설치도, 포트 설정도 필요 없었습니다. 상자 안에 전부 들어 있었으니까요.

(이걸 진작 했으면 한 시간을 아꼈을 텐데.)

그날 이후로 환경 세팅에 한 시간 넘게 쓴 적이 없습니다. 이제 이 상자가 어떻게 만들어지는지 직접 살펴보겠습니다.

이 장의 실습 코드는 아래 레포에서 확인할 수 있습니다.

```
git clone https://github.com/kjh5848/spring-docker
git clone https://github.com/kjh5848/spring-app
```

레포	설명
kjh5848/spring-docker	Dockerfile + docker-compose.yml (완성본)
kjh5848/spring-app	Spring Boot 서버 소스 코드

```
kjh5848/spring-docker/
├── Dockerfile          [설명] 이미지 빌드 설정
├── entrypoint.sh       [설명] 컨테이너 시작 시 자동 실행
└── docker-compose.yml  [실습] 서비스 정의 + 포트 매핑

kjh5848/spring-app/
└── SpringDokerController.java  [참고] health check 엔드포인트
```

챕터를 따라 하며 코드를 작성하고, 막히면 완성 코드를 참고하세요.

환경 준비

실습 전에 Docker가 설치되어 있는지 확인합니다.

```
docker --version
docker compose version
```

`Docker version 24.x` 이상, `Docker Compose version v2.x` 이상이 출력되면 준비가 된 것입니다. 설치되어 있지 않다면 Docker Desktop 공식 사이트에서 내려받아 설치합니다.

환경	확인 사항
Apple Silicon (M1/M2/M3)	Docker Desktop 4.25 이상 권장. Rosetta 에뮬레이션 없이 ARM 이미지를 사용합니다
Intel Mac	별도 설정 없이 동작합니다
Windows	WSL 2 백엔드가 활성화되어 있어야 합니다. Docker Desktop 설치 시 자동으로 안내합니다

Docker Desktop이 실행 중인지도 확인합니다. 상태 표시줄(Mac) 또는 시스템 트레이(Windows)에 고래 아이콘이 보이면 실행 중입니다. 실행되어 있지 않으면 `docker ps` 명령이 `Cannot connect to the Docker daemon` 에러를 출력합니다.

그림 1-1: 이번 챕터의 실습 흐름

1.1 도커는 왜 필요한가

로컬에 자바와 MySQL을 직접 설치하면 두 가지 문제가 생깁니다. 첫째, 버전 충돌 입니다. 프로젝트마다 요구하는 자바 버전이 다릅니다. 하나를 맞추면 다른 하나가 깨집니다. 둘째, 환경 차이 입니다. 내 컴퓨터, 동료 컴퓨터, 서버 — 세 곳의 설정이 조금씩 다릅니다. "내 컴퓨터에서는 되는데"가 여기서 나옵니다.

컨테이너(Container) 는 애플리케이션과 실행 환경을 하나로 묶은 격리된 공간입니다. 컨테이너 안에서는 항상 같은 자바 버전, 같은 라이브러리, 같은 설정이 보장됩니다. Docker는 이 컨테이너를 만들고 실행하는 도구입니다.

그림 1-1: Dockerfile에서 이미지를 만들고, 이미지에서 컨테이너를 실행하는 흐름

비유	Docker 용어	설명
상자 설계도	이미지(Image)	실행에 필요한 모든 것을 담은 읽기 전용 템플릿
열어서 쓰는 상자	컨테이너(Container)	이미지를 실행한 인스턴스. 실제로 프로세스가 돌아가는 공간
설계도 작성법	Dockerfile	이미지를 만드는 스크립트

1.2 Dockerfile + entrypoint.sh

Dockerfile은 이미지를 만드는 설계도입니다. "어떤 환경 위에, 무엇을 복사하고, 무엇을 실행할지"를 순서대로 적습니다.

```
FROM eclipse-temurin:21-jdk
RUN apt-get update && apt-get install -y git
WORKDIR /app

COPY ./entrypoint.sh ./entrypoint.sh
RUN chmod +x ./entrypoint.sh
ENTRYPOINT ["/bin/bash", "./entrypoint.sh"]
```

`FROM` 은 베이스 이미지를 지정합니다. eclipse-temurin:21-jdk를 쓰면 자바 21이 설치된 리눅스 환경이 준비됩니다. 로컬에 자바를 설치할 필요가 없는 이유입니다. `RUN` 은 이미지를 빌드할 때 실행할 명령어입니다. 여기서는 패키지 목록을 갱신하고 git을 설치합니다. 컨테이너 안에서 소스 코드를 받아오기 위해 필요합니다. `WORKDIR` 은 이후 명령어가 실행될 작업 디렉토리를 지정합니다. `COPY` 로 entrypoint.sh를 컨테이너 안으로 복사하고, `ENTRYPOINT` 로 컨테이너가 시작될 때 이 스크립트를 실행하도록 설정합니다.

entrypoint.sh는 컨테이너가 시작되면 자동으로 실행되는 스크립트입니다.

```
git clone "https://github.com/kjh5848/spring-app"
cd spring-app
chmod +x ./gradlew
./gradlew clean build
java -jar build/libs/*.jar
```

GitHub에서 소스를 받아오고, Gradle로 빌드하고, 결과물인 jar 파일을 실행합니다. 컨테이너를 시작하기만 하면 소스 클론부터 서버 실행까지 한 번에 일어납니다.

1.3 docker-compose로 Spring 서버 실행

아래 코드를 `docker-compose.yml` 에 작성합니다.

```
services:
  app:
    build:
      context: .
```

```
      dockerfile: Dockerfile
    container_name: spring-docker
    ports:
      - "8080:8080"
```

`build` 는 현재 디렉토리의 Dockerfile을 사용해 이미지를 빌드하라는 뜻입니다. `ports` 의 `"8080:8080"` 은 호스트의 8080 포트를 컨테이너의 8080 포트에 연결합니다. 브라우저에서 `localhost:8080` 으로 접근하면 컨테이너 안의 Spring 서버에 도달합니다.

터미널에서 실행합니다.

```
docker compose up --build
```

빌드가 진행되면서 터미널에 로그가 출력됩니다. 이미지 다운로드, 의존성 설치, Gradle 빌드 과정이 순서대로 지나갑니다.

```
○ nomadlab@nomadeulaeb-ui-MacBookPro spring-docker % docker compose up
 [+] Building 2.6s (13/13) FINISHED
  => [internal] load local bake definitions                                                    0.0s
  => => reading from stdin 557B                                                                0.0s
  => [internal] load build definition from Dockerfile                                          0.0s
  => => transferring dockerfile: 1.63kB                                                        0.0s
  => [internal] load metadata for docker.io/library/eclipse-temurin:21-jdk                    2.0s
  => [auth] library/eclipse-temurin:pull token for registry-1.docker.io                        0.0s
  => [internal] load .dockerignore                                                             0.0s
  => => transferring context: 2B                                                               0.0s
  => [1/5] FROM docker.io/library/eclipse-temurin:21-jdk@sha256:cd772abe6bc42ddc2f5927756ea33fb26470726438fe0631  0.0s
  => => resolve docker.io/library/eclipse-temurin:21-jdk@sha256:cd772abe6bc42ddc2f5927756ea33fb26470726438fe0631  0.0s
  => [internal] load build context                                                             0.0s
  => => transferring context: 569B                                                             0.0s
  => CACHED [2/5] RUN apt-get update && apt-get install -y git                                 0.0s
  => CACHED [3/5] WORKDIR /app                                                                 0.0s
  => [4/5] COPY ./entrypoint.sh ./entrypoint.sh                                                0.0s
  => [5/5] RUN chmod +x ./entrypoint.sh                                                        0.2s
  => exporting to image                                                                        0.1s
  => => exporting layers                                                                       0.0s
  => => exporting manifest sha256:f106dcf98aed7224ab006429384443832f23b5434b26493b52b078f5426fcc179  0.0s
  => => exporting config sha256:9ca429b1c6e1b99e92ef76300cc0c09d28fa47cc0e1d836aa38ef6bfa060278a  0.0s
  => => exporting attestation manifest sha256:262e41be3d10dacef128f08a24b563c18079a7c10774d4c0b6cf61d940a9a5fe  0.0s
  => => exporting manifest list sha256:cd030485a6fb9d41193a58d7a8526e23b9fcb2060e462f5e98441b29b9805498  0.0s
  => => naming to docker.io/library/spring-docker-app:latest                                   0.0s
  => => unpacking to docker.io/library/spring-docker-app:latest                                0.0s
  => resolving provenance for metadata file                                                    0.0s
 [+] Running 3/3
  ✔ spring-docker-app              Built                                                       0.0s
  ✔ Network spring-docker_default  Created                                                     0.0s
  ✔ Container spring-docker        Created                                                     0.1s
 Attaching to spring-docker
 spring-docker  | Cloning into 'spring-app'...
 spring-docker  | Downloading https://services.gradle.org/distributions/gradle-8.14.3-bin.zip
 spring-docker  | ............10%............20%............30%............40%............50%............60%.....
 ........70%............80%............90%............100%
```

docker compose up 실행 — 이미지 빌드 과정

빌드가 끝나면 Spring Boot 시작 로그가 나타납니다. `Started` 메시지가 보이면 서버가 정상적으로 올라온 것입니다.

```
spring-docker  | > Task :check
spring-docker  | > Task :build
spring-docker  |
spring-docker  | BUILD SUCCESSFUL in 31s
spring-docker  | 7 actionable tasks: 6 executed, 1 up-to-date
spring-docker  |
spring-docker  |   .   ____          _            __ _ _
spring-docker  |  /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
spring-docker  | ( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
spring-docker  |  \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
spring-docker  |   '  |____| .__|_| |_|_| |_\__, | / / / /
spring-docker  |  =========|_|==============|___/=/_/_/_/
spring-docker  |
spring-docker  |  :: Spring Boot ::                (v3.5.7)
spring-docker  |
spring-docker  | 2025-12-15T14:41:40.403Z  INFO 1 --- [spring-doker] [           main] c.m.spring_doker.SpringDokerApp
lication  : Starting SpringDokerApplication v0.0.1-SNAPSHOT using Java 21.0.9 with PID 1 (/app/spring-app/build/libs/s
pring-doker-0.0.1-SNAPSHOT.jar started by root in /app/spring-app)
spring-docker  | 2025-12-15T14:41:40.405Z  INFO 1 --- [spring-doker] [           main] c.m.spring_doker.SpringDokerApp
lication  : The following 1 profile is active: "docker"
spring-docker  | 2025-12-15T14:41:40.871Z  INFO 1 --- [spring-doker] [           main] o.s.b.w.embedded.tomcat.TomcatW
ebServer  : Tomcat initialized with port 8080 (http)
spring-docker  | 2025-12-15T14:41:40.881Z  INFO 1 --- [spring-doker] [           main] o.apache.catalina.core.StandardC
Service   : Starting service [Tomcat]
spring-docker  | 2025-12-15T14:41:40.881Z  INFO 1 --- [spring-doker] [           main] o.apache.catalina.core.StandardC
Engine    : Starting Servlet engine: [Apache Tomcat/10.1.48]
spring-docker  | 2025-12-15T14:41:40.906Z  INFO 1 --- [spring-doker] [           main] o.a.c.c.C.[Tomcat].[localhost].
[/]       : Initializing Spring embedded WebApplicationContext
spring-docker  | 2025-12-15T14:41:40.906Z  INFO 1 --- [spring-doker] [           main] w.s.c.ServletWebServerApplicati
onContext : Root WebApplicationContext: initialization completed in 476 ms
spring-docker  | 2025-12-15T14:41:41.109Z  INFO 1 --- [spring-doker] [           main] o.s.b.w.embedded.tomcat.TomcatW
ebServer  : Tomcat started on port 8080 (http) with context path '/'
spring-docker  | 2025-12-15T14:41:41.119Z  INFO 1 --- [spring-doker] [           main] c.m.spring_doker.SpringDokerApp
lication  : Started SpringDokerApplication in 0.947 seconds (process running for 1.197)
```

Spring Boot 서버가 8080 포트에서 시작된 로그

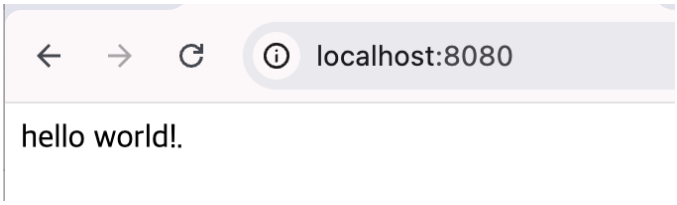
터미널을 새로 열어서 컨테이너 상태를 확인합니다.

```
docker ps
```

`spring-docker` 컨테이너가 `Up` 상태이고 `0.0.0.0:8080->8080/tcp` 포트 매핑이 보이면 성공입니다.

[CAPTURE NEEDED: `docker ps` 실행 결과 - spring-docker 컨테이너가 Up 상태인 터미널 화면. 경로: assets/CH01/terminal/01_docker-ps.png]

이제 브라우저에서 `http://localhost:8080` 에 접속합니다.



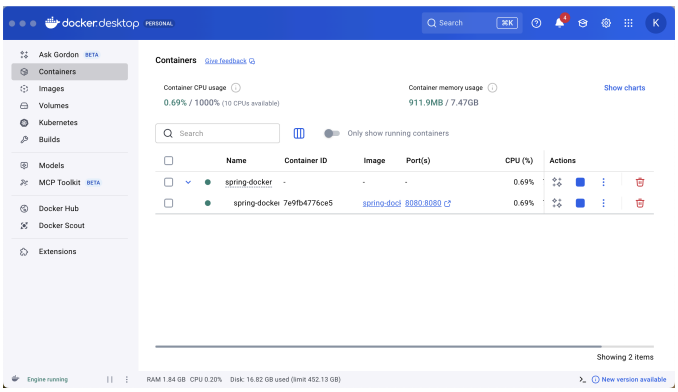
localhost:8080 접속 결과 — hello world!. 텍스트가 표시된 브라우저 화면

`hello world!.` 텍스트가 보이면 Docker 위에서 Spring 서버가 정상 동작하는 것입니다. Spring 서버의 컨트롤러가 응답한 결과입니다.

```
@RestController
public class SpringDokerController {
    @GetMapping("/")
    public String hello() {
        return "hello world!.";
```

```
    }
}
```

Docker Desktop에서도 컨테이너가 실행 중인 것을 확인할 수 있습니다.



Docker Desktop — spring-docker 컨테이너가 실행 중인 화면

자바 설치, 빌드 도구 설정, 포트 설정 — 이 모든 과정이 `docker compose up --build` 한 줄로 끝났습니다. 컨테이너를 내릴 때는 `docker compose down` 을 실행합니다.

```
● nomadlab@nomadeulaeb-ui-MacBookPro spring-docker % docker compose down
  [+] Running 2/2
   ✔ Container spring-docker        Removed
   ✔ Network spring-docker_default  Removed
○ nomadlab@nomadeulaeb-ui-MacBookPro spring-docker %
```

docker compose down 실행 결과 — 컨테이너와 네트워크가 정리된 화면

1.4 Docker 전체 흐름

그림 1-2: Dockerfile 작성부터 브라우저 접속까지의 전체 흐름

1.5 도커 명령어 정리

명령어	설명
`docker compose up --build`	이미지 빌드 후 컨테이너 실행
`docker compose down`	컨테이너 중지 및 삭제
`docker ps`	실행 중인 컨테이너 목록
`docker logs [컨테이너명]`	컨테이너 로그 확인
`docker exec -it [컨테이너명] bash`	실행 중인 컨테이너 안으로 진입
`docker images`	로컬에 저장된 이미지 목록
`docker rmi [이미지명]`	이미지 삭제

`docker compose up` 과 `docker compose down` 두 개만 기억해도 일상적인 개발에는 충분합니다.

이것만은 기억하자

환경을 코드로 관리한다는 것. Dockerfile에 자바 버전을 적고, docker-compose.yml에 포트를 적으면 누가 언제 실행해도 같은 결과가 나옵니다. “내 컴퓨터에서는 되는데”는 환경이 사람마다 다를 때 생기는 말입니다. 환경을 코드로 고정하면 그 말이 사라집니다.

다음 주, 카카오 로그인을 붙여달라는 요청이 옵니다.

2장. 카카오 로그인이 두려웠던 이유

카카오 로그인 붙여주세요

입사 2주차, 자리에 앉자마자 팀장 이 슬랙 메시지를 보냈습니다.

팀장: “이번 스프린트에 카카오 소셜 로그인 붙여주세요. OAuth 2.0 인가 코드 방식으로 하면 돼요.”

(OAuth? 인가 코드? 그게 뭔데.)

검색을 시작했습니다. 화면에 화살표가 잔뜩 그려진 다이어그램이 떴습니다. Resource Owner, Authorization Server, Client, Redirect URI, Access Token, Refresh Token, ID Token, JWKS. 화살표 하나하나에 용어가 붙어 있었고 한 바퀴를 돌 때마다 새로운 단어가 나왔습니다. 스크롤을 내릴수록 머리가 멍해졌습니다.

(로그인이면 아이디 비밀번호 받아서 세션에 넣으면 되는 거 아닌가. 왜 이렇게 복잡하지.)

옆자리 선배 에게 물었습니다.

오픈이: “이거 화살표가 너무 많은데 어디서부터 봐야 돼?”

선배: “그 그림 전체를 이해하려고 하지 마. 우리가 하는 건 딱 하나야. 카카오한테 ‘이 사람 누구야?’ 물어보는 거거든.”

대사관을 떠올려 보겠습니다.

외국에 나가면 내가 누구인지 증명할 방법이 필요합니다. 그래서 대사관에 갑니다. 신청서를 내면 대사관이 여권을 발급해 줍니다. 여권에는 이름, 사진, 유효 기간이 적혀 있습니다. 이후 어느 나라에 가든 이 여권을 보여주면 됩니다. 공항 직원이 대사관에 전화해서 “이 사람 진짜 맞아요?” 확인할 필요가 없습니다. 여권 자체에 위변조 방지 장치가 들어 있으니까요.

OAuth 2.0 이 하는 일이 대사관과 비슷합니다. 우리 서비스는 사용자가 누구인지 직접 확인할 능력이 없습니다. 그래서 카카오라는 대사관에 “이 사람 좀 확인해 주세요”라고 요청합니다. 카카오가 사용자를 확인하고 통행증(Access Token) 을 발급합니다. 우리 서비스는 이 통행증을 들고 카카오에 다시 가서 “이 통행증 가진 사람 이름이 뭐예요?”라고 물어봅니다. 카카오가 이름을 알려주면 로그인이 끝납니다.

여기서 한 가지 불편한 점이 생깁니다. 통행증을 받을 때마다 카카오에 전화해서 이름을 물어봐야 합니다. 사용자가 100명이면 100번 전화합니다. 그래서 카카오가 한 가지를 더 제공합니다. 통행증과 함께 신분증(ID Token) 을 같이 줍니다. 신분증에는 이름이 이미 적혀 있고 위변조 방지 도장까지 찍혀 있습니다. 이 신분증만 있으면 카카오에 다시 전화할 필요가 없습니다. 도장이 진짜인지만 확인하면 됩니다. 이것이 OIDC(OpenID Connect) 입니다.

선배 가 말한 "카카오한테 '이 사람 누구야?' 물어보는 것"이 Access Token으로 사용자 정보를 조회하는 방식이었습니다. 거기에 ID Token을 더하면 전화를 한 번 줄일 수 있습니다. 두 방식을 직접 만들어 보겠습니다.

이 장의 실습 코드는 아래 레포에서 확인할 수 있습니다. start 레포를 클론해서 챕터를 따라 코드를 작성하고, 막히면 end 레포의 완성 코드를 참고하세요.

```
# SSR 방식
git clone https://github.com/metacoding-11-spring-reference/kakao-oauth-code-ssr-start
git clone https://github.com/metacoding-11-spring-reference/kakao-oauth-code-ssr-end

# OIDC 방식
git clone https://github.com/metacoding-11-spring-reference/kakao-oauth-code-oidc-start
git clone https://github.com/metacoding-11-spring-reference/kakao-oauth-code-oidc-end
```

레포	설명
kakao-oauth-code-ssr-start / end	SSR 방식 시작 코드 / 완성 코드
kakao-oauth-code-oidc-start / end	OIDC 방식 시작 코드 / 완성 코드

```
kakao-oauth-code-ssr-start → end/
├── UserController.java        [실습] 카카오 로그인 리다이렉트 + 세션 처리
├── KakaoApiClient.java        [실습] RestTemplate으로 카카오 API 호출
├── KakaoResponse.java         [설명] 토큰 + 사용자 정보 응답 DTO
├── UserService.java           [실습] 토큰 교환 + 회원 저장 로직
└── templates/login.mustache   [참고] 카카오 로그인 버튼 UI

kakao-oauth-code-oidc-start → end/
├── KakaoOidcUtil.java         [실습] JWKS 공개키 조회 + RSA 검증
├── JwtAuthFilter.java         [실습] JWT 인증 필터
├── JwtUtil.java               [실습] JWT 생성 + 검증
└── SecurityConfig.java        [설명] 필터 등록
```

그림 2-1: 이번 챕터의 실습 흐름

2.1 준비 - 카카오 디벨로퍼 앱 설정

카카오 소셜 로그인을 사용하려면 카카오 디벨로퍼 사이트에서 앱을 등록하고 몇 가지 설정을 마쳐야 합니다. 처음 해보면 메뉴가 많아서 헤맬 수 있으니 아래 순서대로 따라 해 보겠습니다.

1단계 - 앱 생성

카카오 디벨로퍼(https://developers.kakao.com/)에 접속해서 로그인합니다.

카카오 디벨로퍼 메인 화면

상단 네비게이션에서 내 애플리케이션 을 클릭하고 앱 추가하기 를 누릅니다.

앱 추가하기 버튼 위치

앱 이름을 입력하고 저장합니다. 이름은 자유롭게 지으면 됩니다.

앱 이름 입력 후 저장

생성된 앱을 클릭해서 대시보드로 이동합니다.

앱 대시보드 - 여기서부터 설정을 시작한다

2단계 - 카카오 로그인 활성화 + OpenID Connect

왼쪽 메뉴에서 제품설정 > 카카오 로그인 > 일반 으로 이동합니다. 사용 설정 을 ON으로 변경하고 OpenID Connect 도 ON으로 활성화합니다.

카카오 로그인 사용 설정 ON + OpenID Connect 활성화 - 두 항목 모두 ON이면 성공이다

3단계 - 동의항목 설정

제품설정 > 카카오 로그인 > 동의항목 으로 이동합니다.

동의항목 페이지 - 개인정보 항목 목록이 표시된다

닉네임 항목의 설정 을 클릭합니다. 동의 단계를 필수 동의 로 변경하고 저장합니다.

닉네임 동의 단계를 필수 동의로 변경

저장 후 닉네임 상태가 필수 동의 로 바뀌었는지 확인합니다.

닉네임이 필수 동의로 변경된 상태

4단계 - Redirect URI 등록

제품설정 > 카카오 로그인 > 일반 에서 아래로 스크롤하면 Redirect URI 항목이 있습니다. 아래 주소를 등록합니다.

```
http://localhost:8080/oauth/callback
```

Redirect URI 등록 - 카카오 로그인 완료 후 되돌아올 우리 서버 주소

5단계 - REST API 키 확인

왼쪽 메뉴에서 앱 설정 > 앱 키 로 이동합니다. REST API 키 를 복사해서 메모장에 저장합니다.

앱 키 페이지 - REST API 키를 복사한다

6단계 - Client Secret 발급

제품설정 > 카카오 로그인 > 보안 에서 Client Secret 코드를 발급받습니다. 발급 후 활성화 상태 를 ON으로 변경합니다. 발급된 코드를 메모장에 저장합니다.

Client Secret 발급 - 활성화 상태가 사용함이면 성공이다

7단계 - .env 파일에 키 등록

start 레포를 클론했으면 프로젝트 루트에 `.env` 파일을 만들고 복사해둔 키를 넣습니다.

```
KAKAO_CLIENT_ID=복사한_REST_API_키
KAKAO_CLIENT_SECRET=복사한_Client_Secret
```

여기까지 완료했으면 카카오 디벨로퍼 설정이 끝났습니다. 설정 요약입니다.

항목	값	확인 방법
REST API 키	앱 키 페이지에서 복사	.env 파일에 저장됨
Client Secret	보안 메뉴에서 발급 + 활성화	.env 파일에 저장됨
Redirect URI	`http://localhost:8080/oauth/callback`	카카오 로그인 > 일반에 등록됨
카카오 로그인	활성화 ON	일반 메뉴에서 확인
OpenID Connect	활성화 ON	일반 메뉴에서 확인
닉네임 동의	필수 동의	동의항목 메뉴에서 확인

2.2 SSR: Code 방식 로그인

OAuth 2.0에는 여러 인증 방식이 있지만 서버 사이드 애플리케이션에서 가장 널리 쓰이는 것은 인가 코드 방식(Authorization Code Grant) 입니다. 흐름을 구성 요소부터 살펴보겠습니다.

비유	OAuth 2.0 용어	설명
여권을 신청하는 사람	리소스 오너(Resource Owner)	카카오 계정을 가진 사용자
대사관	인가 서버(Authorization Server)	카카오 인증 서버. 사용자를 확인하고 토큰을 발급
여행사	클라이언트(Client)	우리 Spring Boot 서버. 카카오에 사용자 정보를 요청
사진이 담긴 서류함	리소스 서버(Resource Server)	카카오 API 서버. 사용자 정보를 보관

인가 코드 방식의 흐름은 두 단계로 나뉩니다. 먼저 인가 코드를 발급받고, 그 코드로 토큰을 교환합니다.

그림 2-2a: 인가 코드 발급 — 사용자가 카카오에 동의하면 인가 코드가 서버로 전달된다

그림 2-2b: 토큰 교환 — 인가 코드로 Access Token을 받고, 사용자 정보를 조회한다

사용자가 "카카오 로그인" 버튼을 누르면 카카오 인증 서버로 이동합니다. 카카오가 로그인과 동의를 받으면 인가 코드(Authorization Code) 를 우리 서버의 Redirect URI로 보냅니다. 우리 서버는 이 인가 코드를 카카오에 다시 보내서 액세스 토큰(Access Token) 으로 교환합니다. 마지막으로 액세스 토큰을 들고 카카오 API에 사용자 정보를 요청합니다.

대사관 비유로 다시 정리하면 이렇습니다. 신청서(인가 코드)를 내면 대사관이 접수증(액세스 토큰)을 줍니다. 접수증을 들고 서류함(리소스 서버)에 가면 사진(사용자 정보)을 받을 수 있습니다. 신청서만으로는 서류함을 열 수 없고 접수증이 있어야 합니다.

인가 코드 요청

사용자가 로그인 버튼을 누르면 카카오 인증 서버로 리다이렉트합니다. 아래 코드를 `UserController.java` 에 작성합니다.

```
@GetMapping("/login/kakao")
public String redirectToKakao() {
    return "redirect:" + userService.카카오로그인주소();
}

@GetMapping("/oauth/callback")
public String kakaoCallback(@RequestParam("code") String code) {
    UserResponse.DTO sessionUser = userService.카카오로그인(code);
    session.setAttribute("sessionUser", sessionUser);
```

```
    return "redirect:/post/list";
}
```

`/login/kakao` 는 카카오 인증 페이지로 보내는 역할만 합니다. 사용자가 카카오에서 동의를 마치면 `/oauth/callback` 으로 인가 코드가 돌아옵니다. 이 코드를 서비스 계층에 넘겨서 토큰 교환부터 사용자 조회까지 처리합니다.

서버를 실행하고 `http://localhost:8080` 에 접속합니다. "카카오로 로그인하기" 링크를 클릭하면 카카오 로그인 화면이 나타납니다.

카카오 로그인 동의 화면 - 닉네임 제공에 동의하면 인가 코드가 발급된다. 이 화면이 보이면 리다이렉트까지 성공이다

동의하고 나면 브라우저 주소창에 `http://localhost:8080/oauth/callback?code=xxxxx` 형태로 인가 코드가 포함된 URL이 표시됩니다. 터미널 로그에서도 code 값을 확인할 수 있습니다.

Access Token 교환 + 사용자 정보 조회

인가 코드를 받았으면 카카오에 보내서 Access Token으로 교환합니다. 아래 코드를 `KakaoApiClient.java` 에 작성합니다.

```
public KakaoResponse.TokenDTO getKakaoToken(String code) {
    HttpEntity<MultiValueMap<String, String>> request = createTokenRequest(code);
    ResponseEntity<KakaoResponse.TokenDTO> response = restTemplate.exchange(
            kakaoTokenUri, HttpMethod.POST, request,
            KakaoResponse.TokenDTO.class);
    return response.getBody();
}
```

`RestTemplate` 으로 카카오 토큰 발급 API를 호출합니다. 요청 본문에는 `grant_type=authorization_code` , `client_id` , `redirect_uri` , `code` , `client_secret` 을 담습니다. 응답으로 돌아오는 `TokenDTO` 에 Access Token이 들어 있습니다.

Access Token을 받았으면 사용자 정보를 조회합니다.

```
public KakaoResponse.KakaoUserDTO getKakaoUser(String accessToken) {
    HttpHeaders headers = new HttpHeaders();
    headers.add("Authorization", "Bearer " + accessToken);
    HttpEntity<MultiValueMap<String, String>> request = new HttpEntity<>(headers);
```

```
    ResponseEntity<KakaoResponse.KakaoUserDTO> response = restTemplate.exchange(
            kakaoUserInfoUri, HttpMethod.GET, request,
            KakaoResponse.KakaoUserDTO.class);
    return response.getBody();
}
```

`Authorization` 헤더에 `Bearer {accessToken}` 을 넣어서 카카오 사용자 정보 API를 호출합니다. 응답에서 닉네임을 꺼내 우리 서비스의 회원으로 등록하거나 기존 회원을 조회합니다.

서버를 다시 실행하고 카카오 로그인을 진행합니다. 동의까지 마치면 터미널에 TokenDTO 응답이 출력됩니다.

Access Token 교환 성공 - 터미널에 accessToken, refreshToken 등이 출력되면 토큰 교환이 정상 동작하는 것이다

세션 로그인 처리

토큰 교환과 사용자 정보 조회를 하나로 묶은 서비스 메서드입니다. `UserService.java` 의 핵심 흐름만 살펴보겠습니다.

```
@Transactional
public UserResponse.DTO 카카오로그인(String code) {
    KakaoResponse.TokenDTO token = kakaoApiClient.getKakaoToken(code);
    KakaoResponse.KakaoUserDTO kakaoUser = kakaoApiClient.getKakaoUser(token.accessToken());
    String username = kakaoUser.properties().nickname();

    User user = userRepository.findByUsername(username)
            .orElseGet(() -> userRepository.save(
                    User.builder()
                        .username(username)
                        .password(UUID.randomUUID().toString())
                        .email("kakao" + kakaoUser.id() + "@kakao.com")
                        .provider("kakao")
                        .build()));
    return new UserResponse.DTO(user);
}
```

인가 코드로 토큰을 받고, 토큰으로 사용자 정보를 받고, 사용자를 DB에 저장하거나 기존 사용자를 찾습니다. 컨트롤러에서 반환된 `UserResponse.DTO` 를 세션에 넣으면 SSR 방식의 카카오 로그인이 완성됩니다.

대사관에 신청서를 내고(인가 코드), 접수증을 받고(Access Token), 접수증으로 서류를 조회하는(사용자 정보) 세 단계가 코드 세 줄에 대응합니다.

서버를 다시 실행하고 카카오 로그인을 진행합니다. 로그인 완료 후 게시글 목록 페이지(`/post/list`)로 이동하면서 상단에 로그인된 사용자 이름이 표시됩니다.

[CAPTURE NEEDED]

SSR 카카오 로그인 성공 - 게시글 목록 페이지에 로그인된 사용자 닉네임이 표시되면 세션 로그인까지 완성이다

2.3 REST API: Code + OIDC 검증

SSR 방식은 잘 동작하지만 한 가지 단점이 있습니다. 사용자 정보를 얻으려면 카카오 API를 한 번 더 호출해야 합니다. 사용자가 몰리면 카카오 API 호출도 그만큼 늘어납니다.

OIDC를 사용하면 이 호출을 줄일 수 있습니다. 토큰 교환 응답에 Access Token과 함께 ID Token 이 포함되기 때문입니다. ID Token 안에 이미 사용자 정보가 들어 있어서 카카오 API를 다시 호출할 필요가 없습니다.

대칭키와 공개키

ID Token을 검증하려면 먼저 대칭키(Symmetric Key) 와 공개키(Public Key) 의 차이를 알아야 합니다.

대칭키는 하나의 열쇠로 잠그고 여는 자물쇠입니다. 보내는 쪽과 받는 쪽이 같은 열쇠를 가지고 있어야 합니다. 우리 서비스 내부에서 JWT를 만들고 검증할 때 사용합니다. 열쇠가 유출되면 누구든 토큰을 만들 수 있으므로 서버 밖으로 나가면 안 됩니다.

공개키는 두 개의 열쇠가 한 쌍인 자물쇠입니다. 하나는 잠그는 용도(개인키), 하나는 여는 용도(공개키)입니다. 카카오가 개인키로 ID Token에 서명하면 우리는 카카오가 공개한 공개키로 서명을 확인합니다. 개인키는 카카오만 가지고 있으므로 ID Token을 위조할 수 없습니다.

비유	키 종류	사용 장면
같은 열쇠로 잠그고 여는 자물쇠	대칭키 (HS256)	우리 서비스 내부 JWT
잠그는 열쇠 / 여는 열쇠가 다른 자물쇠	공개키 (RS256)	카카오 ID Token 검증

OIDC와 ID Token

OIDC(OpenID Connect) 는 OAuth 2.0 위에 "사용자가 누구인지"를 알려주는 계층을 얹은 표준입니다. OAuth 2.0만으로는 "이 사람이 접근 권한이 있다"는 것만 알 수 있습니다. 누구인지는 알 수 없습니다. OIDC는 ID Token 이라는 JWT를 추가로 발급해서 사용자의 식별 정보를 직접 전달합니다.

대사관 비유에서 Access Token이 접수증이라면 ID Token은 여권입니다. 접수증은 서류함에 접근할 권한만 증명합니다. 여권은 이름과 사진이 적혀 있어서 별도로 물어볼 필요가 없습니다. 위변조 방지 도장(공개키 서명)이 찍혀 있으니 도장만 확인하면 됩니다.

그림 2-5a: ID Token 발급 — scope에 openid를 추가하면 ID Token이 함께 응답된다

그림 2-5b: JWKS 검증 — 공개키로 ID Token의 서명을 확인하고 사용자 정보를 추출한다

OIDC를 사용하려면 토큰 요청 시 `scope` 에 `openid` 를 추가합니다. 그러면 응답에 `id_token` 필드가 포함됩니다. 이 ID Token은 JWT 형식이므로 Header, Payload, Signature 세 부분으로 구성됩니다.

Header에는 서명 알고리즘과 `kid` (키 ID)가 들어 있습니다. 이 `kid` 를 카카오의 JWKS(JSON Web Key Set) 엔드포인트에 보내면 해당 공개키를 받을 수 있습니다. 받은 공개키로 서명을 검증하면 ID Token이 카카오가 발급한 진짜인지 확인됩니다.

JWKS 공개키 검증 구현

아래 코드를 `KakaoOidcUtil.java` 에 작성합니다.

```java
public KakaoOidcResponse verify(String idToken) {
    SignedJWT signedJWT = SignedJWT.parse(idToken);
    RSAKey rsaKey = getKeyFromJwks(signedJWT.getHeader().getKeyID());

    if (!signedJWT.verify(new RSASSAVerifier(rsaKey))) {
        throw new RuntimeException("카카오 id_token 서명 검증 실패");
    }
    JWTClaimsSet claims = signedJWT.getJWTClaimsSet();
    return new KakaoOidcResponse(
            claims.getSubject(),
            claims.getStringClaim("nickname"),
            claims.getExpirationTime().toInstant());
}
```

`SignedJWT.parse` 로 ID Token 문자열을 파싱합니다. Header에서 `kid` 를 꺼내 JWKS 엔드포인트에서 RSA 공개키를 조회합니다. `RSASSAVerifier` 로 서명을 검증하고 Payload에서 사용자 식별값(`sub`)과 닉네임을 추출합니다.

공개키를 가져오는 부분도 살펴보겠습니다.

```
private RSAKey getKeyFromJwks(String keyId) {
    JWKSet jwkSet = JWKSet.load(URI.create(kakaoOidcJwksUri).toURL());
    JWK jwk = jwkSet.getKeyByKeyId(keyId);
    if (!(jwk instanceof RSAKey rsaKey)) {
        throw new RuntimeException("RSA 키가 아닙니다: " + keyId);
    }
    return rsaKey;
}
```

`JWKSet.load` 가 카카오의 JWKS 엔드포인트(`https://kauth.kakao.com/.well-known/jwks.json`)에 HTTP 요청을 보내 공개키 목록을 가져옵니다. `kid` 로 해당하는 키를 찾아 RSA 키로 캐스팅합니다. nimbus-jose-jwt 라이브러리가 이 과정을 처리합니다.

Postman에서 OIDC 로그인 흐름을 테스트합니다. 카카오 로그인 → 동의 → 콜백까지 진행하면 응답에 JWT와 사용자 정보가 포함됩니다.

OIDC 로그인 성공 응답 - ID Token 검증 후 사용자 정보와 우리 서비스 JWT가 반환된다. Authorization 헤더에 Bearer {JWT}가 들어 있으면 성공이다

JWT 인증 필터

OIDC로 사용자를 확인한 뒤 우리 서비스용 JWT를 발급합니다. 이후 모든 요청에서 이 JWT를 검증하는 필터가 필요합니다. 아래 코드를 `JwtAuthFilter.java` 에 작성합니다.

```
@Override
protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {
    try {
        jwtProvider.verifyFromHeader(request);
        filterChain.doFilter(request, response);
    } catch (Exception e) {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.setContentType("application/json;charset=UTF-8");
        response.getWriter().write(
                objectMapper.writeValueAsString(
                        new Resp<>(401, e.getMessage(), null)));
    }
}
```

`OncePerRequestFilter` 를 상속해서 모든 요청마다 한 번씩 실행됩니다. `Authorization` 헤더에서 JWT를 꺼내 `JwtProvider` 가 검증합니다. 검증에 실패하면 401 응답을 반환합니다. 로그인 경로 (`/login/kakao` , `/oauth/callback`)는 `shouldNotFilter` 에서 제외합니다.

JWT를 만들고 검증하는 `JwtUtil` 의 핵심 흐름입니다.

```
public static String create(User user) {
    JWTClaimsSet claims = new JWTClaimsSet.Builder()
            .subject(user.getUsername())
            .issueTime(new Date())
            .expirationTime(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
            .claim("id", user.getId())
            .build();
    SignedJWT signedJWT = new SignedJWT(new JWSHeader(JWSAlgorithm.HS256), claims);
    signedJWT.sign(new MACSigner(SECRET));
    return TOKEN_PREFIX + signedJWT.serialize();
}
```

카카오 ID Token은 RSA(공개키)로 서명되어 있었지만 우리 서비스 JWT는 HS256(대칭키)으로 만듭니다. 우리 서버만 만들고 우리 서버만 검증하니까 같은 열쇠 하나면 충분합니다.

2.4 정리

항목	Code 방식 (SSR)	Code + OIDC 방식 (REST API)
토큰 교환	Access Token 발급	Access Token + ID Token 발급
사용자 정보 조회	카카오 API 추가 호출 필요	ID Token 안에 포함 (추가 호출 불필요)
검증 방식	없음 (카카오 API가 대신 검증)	JWKS 공개키로 ID Token 서명 검증
인증 유지	서버 세션	JWT (Authorization 헤더)
적합한 구조	서버 렌더링 (Mustache, Thymeleaf)	프론트-백엔드 분리 (React, Vue)

JWT 심화 주제인 Access Token / Refresh Token 전략은 이 책의 범위를 벗어납니다. 핵심만 정리하면 Access Token은 수명을 짧게(15분~1시간), Refresh Token은 길게(7~30일) 설정해서 Access Token이 만료되면 Refresh Token으로 재발급받는 구조입니다.

비유	OAuth / OIDC 용어	정식 정의
대사관	인가 서버 (Authorization Server)	사용자를 인증하고 토큰을 발급하는 서버
신청서	인가 코드 (Authorization Code)	사용자 동의 후 클라이언트에 전달되는 일회용 코드
접수증	액세스 토큰 (Access Token)	리소스 서버에 접근할 권한을 증명하는 토큰
여권	ID 토큰 (ID Token)	사용자 식별 정보가 담긴 JWT. OIDC에서 발급
위변조 방지 도장	JWKS (JSON Web Key Set)	토큰 서명을 검증할 수 있는 공개키 목록
같은 열쇠 자물쇠	대칭키 (Symmetric Key)	암호화와 복호화에 같은 키를 사용하는 방식
열쇠 두 개 자물쇠	공개키 (Public Key)	서명(개인키)과 검증(공개키)에 다른 키를 사용하는 방식

이것만은 기억하자

소셜 로그인은 결국 토큰 교환입니다. 인가 코드를 보내면 토큰이 돌아오고 토큰으로 사용자를 확인합니다. OAuth 2.0은 “권한을 위임”하는 표준이고 OIDC는 그 위에 “누구인지”를 알려주는 계층을 얹은 것입니다. 화살표가 아무리 많아도 우리 서버가 하는 일은 “카카오한테 물어보고 답을 받는 것” 하나입니다.

다음 주, 서버가 2대가 되면서 로그인이 풀리기 시작합니다.

3장. 서버가 늘었는데 로그인이 풀립니다

서버 2대의 배신

카카오 로그인을 붙인 서비스가 잘 돌아가고 있었습니다. 사용자가 조금씩 늘자 팀장 이 슬랙을 보냈습니다.

팀장: “트래픽이 좀 늘고 있어요. 서버 한 대 더 띄워주세요.”

(서버 한 대 더? Docker Compose에 서비스 하나 복사하면 되겠지.)

어렵지 않은 일이었습니다. 서버를 2대로 늘리고 Nginx를 앞에 세워서 요청을 나눴습니다. 배포를 마치고 모니터를 바라봤습니다. 잘 돌아가는 것 같았습니다.

30분 뒤 선배 가 고개를 돌렸습니다.

선배: “나 방금 로그인했는데 페이지 새로고침하니까 다시 로그인하래.”

(뭐? 방금 배포한 거밖에 없는데.)

직접 확인해 봤습니다. 로그인하고 페이지를 몇 번 왔다 갔다 하니까 갑자기 “누구세요?”라는 에러가 떴습니다. 다시 로그인하면 또 잠깐 되다가 또 풀렸습니다. 규칙도 없이 무작위로 풀리는 것 같았습니다.

(서버를 늘렸을 뿐인데 왜 로그인이 풀리지.)

음식점을 떠올려 보겠습니다.

단골 식당에 갔습니다. 사장님이 혼자 주문도 받고 음식도 만듭니다. “저 매운 거 빼주세요”라고 한마디 하면 사장님이 기억해 둡니다. 다음에 와도 “아 매운 거 빼는 분”이라고 알아봅니다. 손님이 한 명이든 열 명이든 사장님 머릿속에 전부 들어 있습니다.

그런데 장사가 잘 돼서 직원을 한 명 뽑았습니다. 이제 사장님과 직원이 번갈아 주문을 받습니다. 문제는 “매운 거 빼주세요”라고 한 정보가 사장님 머릿속에만 있다는 것입니다. 직원이 주문을 받으면 매

운 거 빼달라는 걸 모릅니다. 손님 입장에서는 “아까 말했잖아요”인데 직원 입장에서는 처음 듣는 이야기입니다.

서버 세션이 이 사장님 머릿속과 같습니다. 서버가 1대일 때는 모든 요청이 같은 서버로 가니까 세션 정보가 항상 남아 있습니다. 서버가 2대가 되면 요청이 A로 갔다가 B로 갔다가 합니다. A 서버에서 로그인했는데 다음 요청이 B로 가면 B는 이 사용자를 모릅니다. “누구세요?”가 나옵니다.

해결 방법은 간단합니다. 주문 노트를 카운터에 올려두면 됩니다. 사장님이든 직원이든 카운터 노트를 보면 “매운 거 빼는 분”을 알 수 있습니다. 머릿속이 아니라 공용 장소에 정보를 두는 것입니다.

Redis 가 이 카운터 노트 역할을 합니다. 서버 A와 서버 B가 각자 세션을 들고 있는 대신 Redis라는 외부 저장소에 세션을 넣어둡니다. 어느 서버로 요청이 가든 Redis를 보면 “아 이 사람 로그인한 사람이구나”를 알 수 있습니다.

선배 에게 설명했습니다.

오픈이: “서버마다 세션이 따로 있어서 그래. Redis에 세션을 모으면 돼.”

선배: “아 그러면 서버가 10대가 돼도 상관없겠네.”

맞습니다. 서버가 몇 대든 세션이 한 곳에 있으면 로그인은 풀리지 않습니다. 이제 직접 만들어 보겠습니다.

이 장의 실습 코드는 아래 레포에서 확인할 수 있습니다.

```
git clone https://github.com/metacoding-11-spring-reference/docker-session-share
```

```
docker-session-share/
├── docker-compose.yml                [실습] 전체 인프라 구성
├── nginx/nginx.conf                  [설명] 경로 기반 라우팅
├── app1/
│   ├── application.properties        [실습] Redis 세션 설정
│   ├── build.gradle                  [실습] 의존성 추가
│   ├── HomeController.java           [설명] 세션 카운터 로직
│   └── templates/index.mustache      [참고] 서버명 + 카운트 표시
└── app2/
    └── (app1과 동일 구조)             [참고]
```

그림 3-1: Redis 세션 공유 아키텍처

환경 준비

이번 장의 docker-compose.yml은 4개 서비스를 정의합니다.

서비스	역할
redis	세션 저장소. 서버 A와 B가 공유하는 외부 메모리
app1	Spring Boot 서버 A. `SERVER_NAME=App1-Server`
app2	Spring Boot 서버 B. `SERVER_NAME=App2-Server`
nginx	리버스 프록시. URL 경로에 따라 app1 또는 app2로 요청을 분배

Nginx 설정 파일은 `nginx/nginx.conf` 에 위치합니다. 이 파일이 `/app1` 경로를 서버 A로, `/app2` 경로를 서버 B로 보내는 라우팅 규칙을 담고 있습니다. docker-compose.yml의 `volumes` 설정으로 컨테이너 안의 Nginx가 이 파일을 읽습니다.

3.1 전체 아키텍처

클라이언트의 요청이 서비스에 도착하기까지의 흐름입니다.

그림 3-1: 전체 아키텍처 - 클라이언트 요청은 Nginx를 거쳐 서버 A 또는 B로 전달되고 세션은 Redis에 저장된다

요청 흐름은 네 단계입니다.

순서	구간	설명
1	Client -> Nginx	브라우저가 `localhost` 로 요청
2	Nginx -> Spring A 또는 B	경로(`/app1` , `/app2`)에 따라 서버 분기
3	Spring -> Redis	세션 조회/저장
4	Spring -> Client	응답 반환

그림 3-2: 서버 A, B 역할 - 두 서버는 동일한 애플리케이션이며 Redis를 통해 세션을 공유한다

서버 A와 서버 B는 완전히 같은 코드입니다. 다른 점은 `SERVER_NAME` 환경 변수뿐입니다. 어느 서버에서 응답했는지 화면에 표시하기 위한 값입니다.

3.2 Spring Session + Redis 설정

Spring Boot가 세션을 Redis에 저장하도록 설정합니다. 먼저 의존성을 추가합니다. 아래 코드를 `build.gradle` 에 작성합니다.

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-mustache'
    implementation 'org.springframework.boot:spring-boot-starter-data-redis'
    implementation 'org.springframework.session:spring-session-data-redis'
}
```

`spring-boot-starter-data-redis` 는 Redis 연결을 담당하고 `spring-session-data-redis` 는 서블릿 세션을 Redis로 대체합니다. 이 두 줄만 추가하면 Spring Session 이 자동으로 활성화됩니다.

다음은 Redis 연결 정보입니다. 아래 내용을 `application.properties` 에 작성합니다.

```
spring.session.store-type=redis
spring.data.redis.host=${SPRING_REDIS_HOST:redis}
spring.data.redis.port=${SPRING_REDIS_PORT:6379}
```

`store-type=redis` 한 줄이 핵심입니다. 이 설정만으로 `HttpSession` 의 저장소가 서버 메모리에서 Redis로 바뀝니다. `SPRING_REDIS_HOST` 는 Docker 환경에서 컨테이너 이름으로 주입됩니다.

그림 3-3: 세션 공유 코드 흐름 - application.properties 설정만으로 세션 저장소가 Redis로 전환된다

세션에 방문 횟수를 저장하는 컨트롤러의 핵심 흐름입니다.

```
@GetMapping("/")
public String home(HttpServletRequest request, Model model) {
    HttpSession session = request.getSession();
    Integer count = (Integer) session.getAttribute("count");
    count = (count == null) ? 1 : count + 1;
    session.setAttribute("count", count);

    model.addAttribute("serverName", serverName);
    model.addAttribute("count", count);
    return "index";
}
```

`request.getSession()` 이 반환하는 세션 객체가 이미 Redis에 연결되어 있습니다. `getAttribute` 로 읽고 `setAttribute` 로 쓰면 Redis에 자동으로 반영됩니다. 기존 코드를 한 줄도 바꾸지 않아도 됩니다.

3.3 Nginx 경로 기반 라우팅

Nginx가 URL 경로를 보고 요청을 서버 A 또는 B로 보냅니다.

그림 3-4: Nginx 라우팅 - `/app1` 경로는 서버 A로, `/app2` 경로는 서버 B로 전달된다

```
events { worker_connections 1024; }

http {
    upstream app1_upstream { server app1:8080; }
    upstream app2_upstream { server app2:8080; }

    server {
        listen 80;
        location /app1 { proxy_pass http://app1_upstream/; }
        location /app2 { proxy_pass http://app2_upstream/; }
        location = / { return 302 /app1/; }
        proxy_redirect off;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

`upstream` 은 "이 이름으로 요청하면 이 서버로 보내라"는 별칭입니다. `/app1` 으로 들어온 요청은 `app1:8080` 으로, `/app2` 는 `app2:8080` 으로 전달됩니다. 루트 경로(`/`)는 `/app1/` 으로 리다이렉트합니다. `proxy_set_header` 는 원래 요청의 호스트와 IP 정보를 뒷단 서버에 전달합니다.

3.4 docker-compose 구성

전체 인프라를 하나의 파일로 정의합니다. 아래 코드를 `docker-compose.yml` 에 작성합니다.

그림 3-5: docker-compose 구성 - Nginx, Spring A, Spring B, Redis 4개 컨테이너가 하나의 네트워크에서 동작한다

```
services:
  nginx:
    image: nginx:latest
```

```yaml
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - app1
      - app2

  app1:
    build: ./app1
    environment:
      - SERVER_NAME=App1-Server
      - SPRING_REDIS_HOST=redis
    depends_on:
      - redis

  app2:
    build: ./app2
    environment:
      - SERVER_NAME=App2-Server
      - SPRING_REDIS_HOST=redis
    depends_on:
      - redis

  redis:
    image: redis:latest
    ports:
      - "6379:6379"
```

컨테이너는 4개입니다. Nginx가 앞에서 요청을 받고 app1과 app2가 처리하며 redis가 세션을 저장합니다. `depends_on` 으로 실행 순서를 지정합니다. Redis가 먼저 뜨고 Spring 앱이 뜨고 Nginx가 마지막에 뜹니다. `SPRING_REDIS_HOST=redis` 는 Docker 네트워크 안에서 redis 컨테이너를 호스트명으로 찾으라는 뜻입니다.

3.5 실행 & 세션 공유 확인

모든 파일이 준비되었으면 실행합니다.

```bash
docker-compose up --build
```

빌드가 끝나면 4개 컨테이너가 모두 올라왔는지 확인합니다. 터미널을 새로 열고 다음을 실행합니다.

```
docker ps
```

`redis`, `nginx`, `app1`, `app2` 4개 컨테이너가 모두 `Up` 상태이면 성공입니다.

[CAPTURE NEEDED: `docker ps` 실행 결과 - redis, nginx, app1, app2 4개 컨테이너가 Up 상태인 터미널 화면. 경로: assets/CH03/terminal/10_docker-ps.png]

브라우저에서 확인합니다. 먼저 `localhost/app1/` 에 접속합니다.

그림 3-6: /app1 접속 - App1-Server에서 응답하고 count가 1이다

화면에 App1-Server 와 count: 1 이 표시됩니다. 서버 A가 응답했고 첫 번째 방문입니다.

이번에는 `localhost/app2/` 에 접속합니다.

그림 3-7: /app2 접속 - App2-Server에서 응답하지만 count가 2로 이어진다

서버가 App2-Server 로 바뀌었는데 count가 2입니다. 서버가 달라졌는데도 이전 방문 기록이 남아 있습니다. Redis에 세션이 저장되어 있기 때문입니다.

다시 `localhost/app1/` 에 접속합니다.

그림 3-8: 다시 /app1 접속 - count가 3으로 이어진다. 서버가 바뀌어도 세션이 유지된다

count가 3입니다. 서버 A -> 서버 B -> 서버 A로 요청이 오갔지만 세션이 끊기지 않았습니다.

Redis CLI로 세션이 실제로 저장되어 있는지 직접 확인할 수 있습니다.

```
docker exec -it redis redis-cli keys '*'
```

`spring:session:sessions:` 로 시작하는 키가 보이면 세션이 Redis에 저장된 것입니다.

[CAPTURE NEEDED: Redis CLI에서 `keys '*'` 실행 결과 - spring:session 키가 표시된 터미널 화면. 경로: assets/CH03/terminal/11_redis-keys.png]

그림 3-9: Redis 세션 공유 정리 - 서버가 여러 대여도 세션을 외부 저장소에 두면 로그인이 유지된다

식당 비유로 돌아가면 사장님과 직원이 카운터 노트를 함께 보는 상태가 된 것입니다. 누가 주문을 받든 “매운 거 빼는 분”을 알 수 있습니다.

비유	기술 용어	정식 정의
사장님 머릿속	서버 메모리 세션	개별 서버 JVM 내부에 저장되는 HttpSession
카운터 노트	Redis	인메모리 키-값 저장소. 세션, 캐시 등에 사용
노트에 적기	Spring Session	HttpSession 구현체를 외부 저장소로 대체하는 프로젝트
직원 배치표	Nginx 리버스 프록시	클라이언트 요청을 뒷단 서버로 분배하는 중간 서버
가게 확장	스케일 아웃 (Scale-Out)	서버 대수를 늘려 처리 능력을 확장하는 방식

이것만은 기억하자

서버를 늘리면 상태를 외부로 분리해야 합니다. 세션이 서버 메모리에 있으면 요청이 다른 서버로 갈 때 로그인이 풀립니다. Redis에 세션을 저장하면 서버가 몇 대든 같은 세션을 공유할 수 있습니다. Spring Session과 Redis 의존성 두 줄, `store-type=redis` 설정 한 줄이면 기존 코드 변경 없이 전환됩니다. 다음 장에서는 프로필 사진을 넣어달라는 요청이 옵니다.

4장. 프로필 사진 좀 넣어주세요

사진 한 장이 이렇게 어려울 줄은

서비스가 안정되자 팀장 이 새로운 요청을 보냈습니다.

팀장: “사용자 프로필에 사진 넣을 수 있게 해주세요. 간단하죠?”

(간단하다고요?)

회원가입, 로그인, 세션까지는 전부 텍스트였습니다. 이름, 이메일, 비밀번호. 전부 글자입니다. 그런데 사진은 글자가 아닙니다. 파일입니다. 서버에 글자를 보내는 건 해봤는데 파일을 보내는 건 해본 적이 없었습니다.

(JSON으로 이름이랑 이메일 보내듯이 사진도 보내면 안 되나? 근데 사진은 JSON이 아닌데.)

선배 에게 물어봤습니다.

오픈이: “이미지를 서버에 보내려면 어떻게 해?”

선배: “Multipart로 보내거나 Base64로 인코딩해서 보내거나.”

오픈이: “Base64가 뭔데?”

선배: “사진을 글자로 바꾸는 거야.”

(사진을 글자로?)

택배를 보내본 적 있으면 이해가 빠릅니다.

친구에게 케이크를 보내고 싶습니다. 그런데 택배 규정이 “상자 안에는 서류만 넣을 수 있습니다”라고 되어 있습니다. 케이크는 서류가 아닙니다. 그냥 넣으면 접수가 안 됩니다.

방법이 하나 있습니다. 케이크를 사진으로 찍고 레시피를 적어서 서류 형태로 만드는 것입니다. 받는 사람이 그 서류를 보고 케이크를 다시 만들면 됩니다. 원본 케이크와 똑같은 것이 도착합니다.

Base64 가 이 과정과 같습니다. 이미지 파일은 바이너리 데이터입니다. JSON은 텍스트만 담을 수 있습니다. 바이너리를 그냥 JSON에 넣으면 깨집니다. 그래서 이미지를 텍스트 형태로 변환(인코딩)해서

JSON에 담고 서버에 보냅니다. 서버는 받은 텍스트를 다시 이미지 파일로 변환(디코딩)해서 저장합니다.

케이크를 서류로 바꿔 보내고 받는 쪽에서 다시 케이크로 복원하는 것입니다.

선배 에게 다시 말했습니다.

오픈이: "아 그러면 프론트에서 이미지를 텍스트로 바꿔서 JSON으로 보내면 되는 거네."

선배: "맞아. 서버에서 다시 파일로 복원해서 저장하면 끝이야."

(텍스트로 바뀌면 지금까지 하던 것처럼 JSON으로 주고받을 수 있겠다.)

이제 직접 만들어 보겠습니다.

이 장의 실습 코드는 아래 레포에서 확인할 수 있습니다.

```
git clone https://github.com/metacoding-11-spring-reference/spring-base64
```

```
spring-base64/
├── ImageController.java      [참고] 엔드포인트 정의
├── ImageService.java         [실습] Base64 디코딩 + UUID + 저장
├── ImageEntity.java          [설명] 엔티티 구조
├── ImageRequest.java         [설명] 요청 DTO (fileName, fileData)
├── ImageResponse.java        [참고] 응답 DTO
├── ImageRepository.java      [참고] JPA Repository
├── WebConfig.java            [설명] 정적 리소스 매핑
├── CorsConfig.java           [설명] CORS 설정
└── application.properties    [참고] 정적 리소스 경로

react-base64/                 [참고] 프론트 연동 확인용
└── (React 프로젝트)
```

React 프론트엔드 코드는 별도 레포에 있습니다.

```
git clone https://github.com/metacoding-11-spring-reference/react-base64
```

그림 4-1: Base64 이미지 업로드 흐름

4.1 전체 흐름 한눈에 보기

이미지 업로드의 전체 흐름입니다.

그림 4-1: 업로드 전체 흐름 - Postman이 Base64 문자열을 보내면 Spring이 파일로 복원하고 DB에 기록한 뒤 URL을 반환한다

순서	구간	설명
1	Postman -> Spring	Base64로 인코딩된 이미지를 JSON에 담아 전송
2	Spring 내부	Base64 디코딩 -> 바이트 배열 -> 파일 저장
3	Spring -> DB	파일 경로, UUID, 파일명을 엔티티로 저장
4	Spring -> Postman	저장된 이미지 정보를 JSON으로 반환

택배 비유로 보면 1번이 "서류로 바꿔서 보내기"이고 2번이 "받아서 케이크로 복원"입니다. 3번은 복원한 케이크가 어디에 있는지 기록하는 것이고 4번은 "잘 받았습니다" 영수증입니다.

4.2 엔티티 + DTO 설계

이미지 정보를 저장할 엔티티입니다.

그림 4-2: ImageEntity 구조 - id, uuid, fileName, url, createdAt 5개 필드로 구성된다

```
@Entity
@Table(name = "image_tb")
public class ImageEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String uuid;
    private String fileName;
    private String url;
    private LocalDateTime createdAt;
}
```

`uuid` 는 파일명 충돌을 방지합니다. 사용자 10명이 동시에 `profile.png` 를 올려도 서버에 저장되는 파일명은 전부 다릅니다. `url` 은 브라우저에서 이미지에 접근할 수 있는 경로입니다.

요청과 응답을 담는 DTO입니다.

그림 4-3: DTO 설계 - 요청은 파일명과 Base64 문자열을 받고 응답은 저장된 이미지 정보를 반환한다

```
public record UploadDTO(
    String fileName,
```

```
    String fileData
){}
```

`fileName` 은 원본 파일명이고 `fileData` 는 Base64로 인코딩된 이미지 문자열입니다. 택배 비유에서 `fileName` 이 "초코 케이크"라는 이름이고 `fileData` 가 서류로 바꾼 레시피입니다.

응답 DTO는 엔티티의 정보를 그대로 돌려줍니다.

```
public record DTO(
    Long id, String uuid, String fileName,
    String url, LocalDateTime createdAt
) {
    public static DTO fromEntity(ImageEntity imageEntity) {
        return new DTO(
            imageEntity.getId(), imageEntity.getUuid(),
            imageEntity.getFileName(), imageEntity.getUrl(),
            imageEntity.getCreatedAt()
        );
    }
}
```

`fromEntity` 는 엔티티를 DTO로 변환하는 정적 팩토리 메서드입니다. 컨트롤러가 엔티티를 직접 반환하지 않고 DTO로 감싸서 반환합니다.

4.3 업로드 API 구현

이 장의 핵심입니다. Base64 문자열을 받아서 파일로 복원하고 저장하는 과정입니다.

그림 4-4: 업로드 API 흐름 - Base64 디코딩, UUID 생성, 파일 저장, DB 기록 순서로 진행된다

아래 코드를 `ImageService.java` 에 작성합니다.

```
byte[] fileBytes = Base64.getDecoder().decode(uploadDTO.fileData());

String uuid = UUID.randomUUID().toString();
int dotIndex = uploadDTO.fileName().lastIndexOf('.');
String ext = uploadDTO.fileName().substring(dotIndex + 1).toLowerCase();
String savedFileName = uuid + "." + ext;

Path filePath = Paths.get("uploads").resolve(savedFileName);
Files.write(filePath, fileBytes);
```

```
String publicUrl = "/uploads/" + savedFileName;
imageRepository.save(ImageEntity.builder()
    .uuid(uuid).fileName(savedFileName)
    .url(publicUrl).createdAt(LocalDateTime.now()).build());
```

`Base64.getDecoder().decode()` 가 텍스트를 바이트 배열로 되돌립니다. 택배로 온 서류를 케이크로 복원하는 단계입니다. `UUID.randomUUID()` 로 세상에 하나뿐인 파일명을 만들고 원본 파일의 확장자를 붙입니다. `Files.write()` 로 서버 디스크에 저장하고 `imageRepository.save()` 로 어디에 저장했는지를 DB에 기록합니다.

컨트롤러는 이 서비스를 호출합니다.

```
@PostMapping("/api/images/upload")
public ImageResponse.DTO upload(
        @RequestBody ImageRequest.UploadDTO uploadDTO) {
    return imageService.upload(uploadDTO);
}
```

`@RequestBody` 가 JSON 본문을 `UploadDTO` 로 변환합니다. 텍스트 형태로 온 이미지가 서비스 레이어에서 파일로 복원됩니다.

Postman으로 테스트해 보겠습니다. 먼저 이미지를 Base64 문자열로 변환해야 합니다.

그림 4-5: Base64 인코딩 - 온라인 변환 사이트에서 이미지를 업로드하면 Base64 문자열이 생성된다

그림 4-6: 인코딩 결과 - 이미지가 긴 텍스트 문자열로 변환된다

그림 4-7: 문자열 복사 - 이 문자열을 Postman의 JSON body에 붙여넣는다

변환된 문자열을 Postman에서 JSON에 담아 보냅니다.

그림 4-8: Postman 업로드 요청 - fileName과 fileData를 JSON으로 전송한다

그림 4-9: 요청 Body - fileData에 Base64 문자열이 담겨 있다

그림 4-10: 업로드 응답 - id, uuid, fileName, url, createdAt이 반환된다

응답 상태 코드가 `200 OK` 이고 본문에 `id` , `uuid` , `fileName` , `url` , `createdAt` 이 모두 보이면 성공입니다.

서버의 `uploads/` 디렉토리에 실제 파일이 생성되었는지 확인합니다.

```
ls uploads/
```

[CAPTURE NEEDED: 터미널에서 ls uploads/ 실행 결과 - UUID가 붙은 이미지 파일(예: 3a7f2c1d-····-profile.png)이 한 개 표시된다]

ls uploads/ 실행 결과 - UUID가 붙은 파일명이 한 개 이상 보이면 파일 저장이 정상 동작한 것이다

응답에 `url` 이 `/uploads/UUID.png` 형태로 돌아옵니다. 이 경로로 브라우저에서 이미지를 볼 수 있습니다.

그림 4-11: 브라우저 확인 - 반환된 URL로 접속하면 업로드한 이미지가 표시된다

브라우저 주소창에 `http://localhost:8080/uploads/UUID.png` (응답의 `url` 값)을 입력했을 때 업로드한 이미지가 그대로 표시되면 성공입니다. 여기까지 확인되면 Base64 디코딩, 파일 저장, 정적 리소스 제공이 모두 동작하는 것입니다.

4.4 정적 리소스 매핑 + 조회 API

업로드된 이미지를 브라우저에서 바로 볼 수 있으려면 Spring이 `uploads/` 폴더를 정적 리소스로 제공해야 합니다.

그림 4-12: 정적 리소스 매핑 - `/uploads/**` 요청이 서버의 uploads 폴더로 연결된다

```
@Override
public void addResourceHandlers(ResourceHandlerRegistry registry) {
    registry.addResourceHandler("/uploads/**")
            .addResourceLocations("file:uploads/");
}
```

`/uploads/**` 로 들어오는 요청을 프로젝트 루트의 `uploads/` 폴더에서 찾으라는 설정입니다. 브라우저가 `/uploads/abc.png` 를 요청하면 서버의 `uploads/abc.png` 파일을 반환합니다.

`application.properties` 에도 정적 리소스 위치를 지정합니다.

```
spring.web.resources.static-locations=file:uploads/
```

목록 조회와 단건 조회 API입니다.

```
@GetMapping("/list")
public List<ImageResponse.DTO> getAllImages() {
    return imageService.listAll();
}
```

```
@GetMapping("/{id}")
public ImageResponse.DTO getImageDetail(
        @PathVariable Long id) {
    return imageService.findById(id);
}
```

`/list` 는 저장된 모든 이미지 정보를 반환하고 `/{id}` 는 특정 이미지 한 건을 반환합니다.

그림 4-13: 단건 조회 - id로 특정 이미지 정보를 조회한다

그림 4-14: 목록 조회 - 저장된 모든 이미지 목록이 반환된다

그림 4-15: 브라우저 이미지 확인 - 정적 리소스 매핑을 통해 이미지가 브라우저에 표시된다

그림 4-16: 정적 리소스 확인 - 서버의 uploads 폴더에 저장된 파일이 브라우저에서 직접 접근 가능하다

4.5 React 연동

프론트엔드에서 이미지를 업로드하려면 CORS(Cross-Origin Resource Sharing) 설정이 필요합니다. 브라우저는 보안상 다른 출처의 서버에 요청을 보내는 것을 기본적으로 막습니다. React 개발 서버는 `localhost:5173` 에서 돌아가고 Spring은 `localhost:8080` 에서 돌아가므로 포트가 다릅니다. 서로 다른 출처입니다.

```
config.addAllowedOrigin("http://localhost:5173");
config.addAllowedMethod("*");
config.addAllowedHeader("*");
config.setAllowCredentials(true);
```

`addAllowedOrigin` 으로 React 개발 서버의 주소를 허용합니다. `*` 은 모든 HTTP 메서드와 헤더를 허용한다는 뜻입니다.

CORS를 설정한 뒤 React에서 이미지를 업로드하고 조회할 수 있습니다.

그림 4-17: React 이미지 업로드 - 파일을 선택하면 Base64로 변환되어 서버에 전송된다

그림 4-18: React 이미지 목록 - 서버에 저장된 이미지들이 리스트로 표시된다

그림 4-19: React 이미지 상세 - 개별 이미지의 정보와 원본 이미지가 표시된다

React 프론트엔드 코드는 `https://github.com/metacoding-11-spring-reference/react-base64` 레포에서 확인할 수 있습니다.

비유	기술 용어	정식 정의
서류만 보낼 수 있는 택배	JSON	텍스트 기반 데이터 교환 형식. 바이너리 데이터를 직접 담을 수 없다
케이크를 서류로 바꾸기	Base64 인코딩	바이너리 데이터를 ASCII 문자열로 변환하는 인코딩 방식
서류를 케이크로 복원	Base64 디코딩	Base64 문자열을 원본 바이너리 데이터로 되돌리는 과정
세상에 하나뿐인 이름표	UUID	Universally Unique Identifier. 충돌 확률이 극히 낮은 고유 식별자
케이크 보관 창고	uploads 폴더	서버 로컬 디스크에 파일을 저장하는 디렉토리
보관 위치 기록장	DB 엔티티	파일 메타데이터(경로, 이름, 생성일)를 데이터베이스에 저장하는 객체

이것만은 기억하자

이미지도 결국 문자열입니다. Base64로 인코딩하면 어떤 바이너리 파일이든 텍스트가 됩니다. 텍스트가 되면 JSON에 담을 수 있고 기존의 `@RequestBody` 방식 그대로 서버에 보낼 수 있습니다. 서버에서는 디코딩해서 파일로 복원하고 UUID로 이름을 붙여 저장합니다. 정적 리소스 매핑까지 설정하면 브라우저에서 바로 이미지를 확인할 수 있습니다.

다음 장에서는 이미지가 쌓이면서 서버 디스크가 가득 차는 문제를 만납니다.

5장. 서버가 다 할 필요는 없잖아요

디스크 사용량 90%

월요일 아침, 모니터링 알림이 울렸습니다.

팀장: "서버 디스크 90% 찼대. 이미지 때문인 것 같은데 좀 봐줘."

(90%?)

지난 장에서 프로필 사진 업로드를 만들 때 이미지를 서버 로컬 디스크에 저장했습니다. uploads 폴더에 파일이 쌓이는 구조였습니다. 서비스를 오픈한 지 두 달밖에 안 됐는데 벌써 디스크가 가득 차고 있었습니다.

(서버 한 대에 이미지를 직접 저장하면 언젠가 이런 일이 생기긴 하지.)

선배 에게 물어봤습니다.

오픈이: "이미지 저장소를 밖으로 빼야 할 것 같은데 어떻게 해?"

선배: "클라우드 스토리지 써. AWS S3 같은 거. 그리고 Presigned URL 쓰면 서버가 파일을 직접 받지도 않아."

오픈이: "서버가 파일을 안 받는다고?"

선배: "서버는 허가증만 발급해. 파일은 클라이언트가 직접 S3에 올려."

(서버가 파일을 안 만지고 허가증만 준다?)

이사를 한 번이라도 해봤으면 이해가 빠릅니다.

원래 집에는 창고가 있었습니다. 택배가 오면 직접 받아서 창고에 넣었습니다. 그런데 물건이 너무 많아져서 창고가 터지기 직전입니다. 그래서 외부 창고를 빌리기로 했습니다.

보통은 택배를 받아서 외부 창고까지 직접 들고 가야 합니다. 그런데 더 좋은 방법이 있습니다. 외부 창고 회사에 전화해서 "오늘 택배 하나 올 건데 이 출입증 보여주면 3번 선반에 넣어주세요"라고 말하는 것입니다. 택배 기사에게 그 출입증을 건네주면 기사가 직접 외부 창고에 물건을 넣습니다. 집주인은 전화 한 통만 하면 됩니다.

Presigned URL 이 이 출입증입니다. Spring 서버가 AWS S3에 "이 주소로 업로드하면 받아줘"라는 출입증을 요청합니다. 클라이언트는 그 출입증을 들고 S3에 직접 파일을 올립니다. 서버는 파일을 한 번도 만지지 않습니다.

그런데 물건이 창고에 들어간 다음에 한 가지 더 할 일이 있습니다. 원본 사진이 5MB짜리 고해상도라면 목록 화면에서 매번 5MB를 내려받는 건 낭비입니다. 외부 창고에 직원을 한 명 두고 "원본이 들어오면 작은 사이즈로 하나 더 만들어 놓으세요"라고 부탁하는 것입니다. 이 직원이 Lambda 입니다. S3에 이미지가 올라오면 자동으로 리사이즈 버전을 만들어 둡니다.

선배 에게 다시 말했습니다.

오픈이: "그러면 서버는 출입증 발급이랑 결과 기록만 하면 되는 거네."

선배: "맞아. 참고로 이 장은 AWS 계정이 필요해. 프리 티어로 충분하니까 없으면 하나 만들어."

(서버가 모든 걸 직접 할 필요는 없구나.)

이제 직접 만들어 보겠습니다. 이 장은 AWS 계정이 필요한 유일한 장입니다.

이 장의 실습 코드는 아래 레포에서 확인할 수 있습니다. start 레포를 클론해서 따라 작성하고 막히면 end 레포를 참고합니다.

```
git clone https://github.com/metacoding-11-spring-reference/spring-presign-url-start
```

완성 코드입니다.

```
git clone https://github.com/metacoding-11-spring-reference/spring-presign-url-end
```

```
spring-presign-url-start → end/
├── AwsS3Config.java            [실습] AWS S3 클라이언트 설정
├── ImageService.java           [실습] Presigned URL 생성 + Resized URL 저장
├── ImageController.java        [실습] 업로드/조회 엔드포인트
├── ImageEntity.java            [설명] 엔티티 (원본URL + 리사이즈URL)
├── ImageRequest.java           [설명] 요청 DTO
├── ImageResponse.java          [설명] 응답 DTO
├── application.properties      [실습] AWS 접속 정보
└── lambda/
    └── resize_function.py      [설명] Lambda 리사이즈 코드 (Python)
```

이번 챕터의 실습 흐름

5.1 아키텍처 구조

전체 흐름은 5단계입니다.

업로드 단계 - 서버는 URL만 발급하고 파일은 클라이언트가 S3에 직접 올린다

후처리 단계 - Lambda가 자동으로 리사이즈하고 Spring은 결과만 기록한다

순서	구간	설명
1	Client -> Spring	Presigned URL 발급 요청
2	Client -> S3	발급받은 URL로 original/{uuid}.ext에 직접 업로드
3	S3 -> Lambda	ObjectCreated 이벤트 자동 발생
4	Lambda -> S3	리사이즈한 이미지를 resized/{uuid}.jpg로 저장
5	Client -> Spring	/complete 호출로 원본 URL + 리사이즈 URL을 DB에 저장

외부 창고 비유로 보면 1번이 "출입증 발급"이고 2번이 "택배 기사가 직접 창고에 물건 넣기"입니다. 3~4번은 "창고 직원이 자동으로 축소본 만들기"이고 5번은 "집주인에게 보관 완료 보고"입니다.

4장에서는 서버가 파일을 직접 받아서 디코딩하고 저장했습니다. 이 장에서는 서버가 허가증만 발급합니다. 파일은 서버를 거치지 않습니다.

5.2 환경 준비 - AWS 설정

코드를 작성하기 전에 AWS 리소스를 먼저 준비합니다. S3 버킷, IAM 사용자, Lambda 함수를 순서대로 만듭니다. AWS 계정이 없다면 프리 티어로 가입합니다.

S3 버킷 생성

AWS 콘솔에서 S3 서비스로 이동합니다.

1. 버킷 만들기를 클릭합니다.
2. 버킷 이름을 입력합니다. 전 세계에서 유일해야 하므로 프로젝트명을 포함시킵니다.
3. 리전은 ap-northeast-2 (서울) 을 선택합니다.
4. "모든 퍼블릭 액세스 차단"을 해제합니다. 실습용이므로 해제하지만 실서비스에서는 반드시 활성화해야 합니다.
5. 나머지 옵션은 기본값으로 두고 버킷 만들기를 클릭합니다.

[CAPTURE NEEDED: AWS S3 버킷 생성 화면 - 리전 ap-northeast-2, 퍼블릭 액세스 차단 해제 상태] 경로: assets/CH05/terminal/01_aws-s3-bucket-create.png

버킷이 생성되면 내부에 폴더 두 개를 만듭니다.

1. 생성한 버킷을 클릭합니다.
2. 폴더 만들기를 클릭하고 `original` 을 입력합니다. 원본 이미지가 저장될 경로입니다.
3. 같은 방법으로 `resized` 폴더를 만듭니다. Lambda가 리사이즈한 이미지가 저장될 경로입니다.

[CAPTURE NEEDED: S3 버킷 내부 - original, resized 폴더가 생성된 상태] 경로: assets/CH05/terminal/02_s3-folder-structure.png

S3 버킷에 original과 resized 두 개의 폴더를 만든다

S3 버킷 정책 설정

버킷의 권한 탭으로 이동합니다. 버킷 정책 편집을 클릭하고 아래 JSON을 입력합니다.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicAllAccess",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::버킷이름",
        "arn:aws:s3:::버킷이름/*"
      ]
    }
  ]
}
```

`버킷이름` 부분을 자신의 버킷 이름으로 바꿉니다. 변경 사항 저장을 클릭합니다.

S3 CORS 설정

프론트엔드에서 S3에 직접 업로드하려면 CORS 설정이 필요합니다. 같은 권한 탭에서 CORS(Cross-origin 리소스 공유) 편집을 클릭합니다.

```
[
    {
        "AllowedHeaders": ["*"],
        "AllowedMethods": ["PUT", "GET", "HEAD"],
        "AllowedOrigins": ["*"],
        "ExposeHeaders": []
```

```
    }
]
```

실습에서는 모든 Origin을 허용합니다. 실서비스에서는 `AllowedOrigins` 에 실제 도메인만 지정합니다.

[CAPTURE NEEDED: S3 CORS 설정 화면 - JSON 입력 후 저장] 경로: assets/CH05/terminal/26_s3-cors-setting.png

S3 CORS 설정 - 프론트엔드에서 S3에 직접 요청할 수 있도록 CORS를 설정한다

IAM 사용자 생성

AWS 콘솔에서 IAM 서비스로 이동합니다.

1. 왼쪽 메뉴에서 사용자를 클릭합니다.
2. 사용자 생성을 클릭하고 이름을 입력합니다.
3. 다음을 클릭합니다.
4. "직접 정책 연결"을 선택하고 검색창에 `AmazonS3FullAccess` 를 입력합니다.
5. 해당 정책을 체크하고 다음을 클릭합니다.
6. 사용자 생성을 클릭합니다.

[CAPTURE NEEDED: IAM 사용자 생성 화면 - AmazonS3FullAccess 정책 연결 상태] 경로: assets/CH05/terminal/03_iam-user-create.png

IAM 사용자에 AmazonS3FullAccess 정책을 연결한다

Access Key 발급

생성한 사용자를 클릭하고 보안 자격 증명 탭으로 이동합니다.

1. 액세스 키 만들기를 클릭합니다.
2. "로컬 코드"를 선택하고 다음을 클릭합니다.
3. 액세스 키 만들기를 클릭합니다.
4. Access Key 와 Secret Key 가 표시됩니다. .csv 파일을 다운로드해서 보관합니다. 이 화면을 닫으면 Secret Key는 다시 볼 수 없습니다.

[CAPTURE NEEDED: IAM 액세스 키 발급 화면 - Access Key와 Secret Key가 표시된 상태] 경로: assets/CH05/terminal/04_iam-access-key.png

액세스 키 발급 - Access Key와 Secret Key를 발급받아 환경 변수에 설정한다

프로젝트 루트에 `.env` 파일을 만들고 발급받은 키를 입력합니다.

```
CLOUD_AWS_CREDENTIALS_ACCESS_KEY=발급받은_ACCESS_KEY
CLOUD_AWS_CREDENTIALS_SECRET_KEY=발급받은_SECRET_KEY
```

```
CLOUD_AWS_REGION=ap-northeast-2
CLOUD_AWS_S3_BUCKET=버킷이름
```

이 `.env` 파일은 `.gitignore` 에 반드시 추가합니다. 키가 깃허브에 올라가면 AWS에서 자동으로 감지하고 경고 메일을 보냅니다.

Lambda 함수 생성

AWS 콘솔에서 Lambda 서비스로 이동합니다.

1. 함수 생성을 클릭합니다.
2. 함수 이름을 입력합니다. (예: `image-resize-handler`)
3. 런타임은 Python 3.11 을 선택합니다.
4. 기본 실행 역할 변경에서 "기본 Lambda 권한을 가진 새 역할 생성"을 확인합니다.
5. 함수 생성을 클릭합니다.

[CAPTURE NEEDED: Lambda 함수 생성 화면 - Python 3.11 런타임 선택] 경로: assets/CH05/terminal/05_lambda-create.png

Lambda 함수 생성 - Python 3.x 런타임으로 함수를 만든다

Lambda 역할에 S3 권한 추가

Lambda 함수가 S3의 이미지를 읽고 쓰려면 권한이 필요합니다.

1. IAM 서비스로 이동합니다.
2. 왼쪽 메뉴에서 역할을 클릭합니다.
3. Lambda 함수 생성 시 자동으로 만들어진 역할을 찾아 클릭합니다.
4. 권한 추가 > 정책 연결을 클릭합니다.
5. `AmazonS3FullAccess` 를 검색하고 체크한 뒤 권한 추가를 클릭합니다.

Lambda에 Pillow Layer 추가

Lambda에는 이미지 처리 라이브러리인 Pillow가 기본 내장되어 있지 않습니다. Layer로 추가합니다.

1. Lambda 함수 페이지 하단의 Layers 섹션에서 Add a layer를 클릭합니다.
2. "ARN 지정"을 선택합니다.
3. 아래 ARN을 입력합니다.

```
arn:aws:lambda:ap-northeast-2:770693421928:layer:Klayers-p311-Pillow:10
```

4. 확인을 클릭합니다.

[CAPTURE NEEDED: Lambda Layer 추가 화면 - Pillow Layer ARN 입력 상태] 경로: assets/CH05/terminal/16_lambda-layer-add.png

Pillow Layer 추가 - Lambda에 이미지 처리 라이브러리를 Layer로 추가한다

S3 이벤트 트리거 연결

`original/` 폴더에 파일이 올라오면 Lambda가 자동으로 실행되도록 트리거를 설정합니다.

1. Lambda 함수 페이지에서 트리거 추가를 클릭합니다.
2. 소스에서 S3를 선택합니다.
3. 버킷에서 생성한 버킷을 선택합니다.
4. 이벤트 유형은 All object create events 를 선택합니다.
5. Prefix에 `original/` 을 입력합니다.
6. 추가를 클릭합니다.

[CAPTURE NEEDED: Lambda S3 트리거 설정 화면 - ObjectCreated, prefix: original/] 경로: assets/CH05/terminal/06_lambda-trigger.png

S3 트리거 설정 - original/ prefix로 ObjectCreated 이벤트를 연결한다

이 화면이 보이면 환경 준비가 완료된 것입니다. S3 버킷, IAM 사용자, Lambda 함수, 트리거까지 모두 연결되었습니다.

5.3 Spring 서버 구현 1 - Presigned URL 발급

먼저 AWS SDK 의존성을 추가합니다.

```
implementation 'software.amazon.awssdk:s3:2.25.31'
```

AWS S3 클라이언트와 Presigner를 빈으로 등록합니다. 아래 코드를 `AwsS3Config.java` 에 작성합니다.

```
@Bean
public S3Client s3Client() {
    return S3Client.builder()
            .region(Region.of(region))
            .credentialsProvider(StaticCredentialsProvider.create(
                AwsBasicCredentials.create(accessKey, secretKey)))
            .build();
}

@Bean
public S3Presigner s3Presigner() {
    return S3Presigner.builder()
```

```
            .region(Region.of(region))
            .credentialsProvider(StaticCredentialsProvider.create(
                AwsBasicCredentials.create(accessKey, secretKey)))
            .build();
}
```

`S3Client` 는 S3 버킷에 직접 명령을 보내는 클라이언트입니다. `S3Presigner` 는 출입증(Presigned URL)을 만드는 전용 객체입니다. 둘 다 같은 인증 정보를 사용합니다.

`application.properties` 에 AWS 접속 정보를 설정합니다. 아래 코드를 `application.properties` 에 작성합니다.

```
cloud.aws.s3.bucket=${AWS_S3_BUCKET}
cloud.aws.region=${AWS_REGION}
cloud.aws.credentials.access-key=${AWS_ACCESS_KEY}
cloud.aws.credentials.secret-key=${AWS_SECRET_KEY}
```

환경 변수로 관리하면 코드에 키가 노출되지 않습니다. 5.2절에서 만든 `.env` 파일의 값이 여기에 주입됩니다.

환경 변수 설정 - AWS 키를 환경 변수로 관리하면 코드에 노출되지 않는다

이 장의 핵심 코드입니다. Presigned URL을 생성하는 서비스 메서드입니다. 아래 코드를 `ImageService.java` 에 작성합니다.

```
String uuid = UUID.randomUUID().toString();
String ext = reqDTO.fileName()
        .substring(reqDTO.fileName().lastIndexOf('.') + 1);
String key = "original/" + uuid + "." + ext;

PutObjectRequest objectRequest = PutObjectRequest.builder()
        .bucket(bucket).key(key)
        .contentType(reqDTO.contentType()).build();

PresignedPutObjectRequest presignedRequest = presigner
        .presignPutObject(b -> b
            .signatureDuration(Duration.ofMinutes(15))
            .putObjectRequest(objectRequest));
```

`UUID.randomUUID()` 로 고유한 파일명을 만들고 `original/` 폴더 아래에 저장될 경로(key)를 구성합니다. `PutObjectRequest` 는 "이 버킷의 이 경로에 이 타입의 파일을 올릴 것이다"라는 명세서입니다. `presignPutObject()` 가 이 명세서를 기반으로 15분짜리 출입증을 발급합니다. 15분이 지나면 출입증은 자동으로 만료됩니다.

컨트롤러입니다. 아래 코드를 `ImageController.java` 에 작성합니다.

```
@PostMapping("/presigned")
public ImageResponse.PresignedDTO generatePresignedUrl(
        @RequestBody ImageRequest.PresignedDTO reqDTO) {
    return imageService.generatePresignedUrl(reqDTO);
}
```

Postman으로 테스트해 보겠습니다.

Presigned URL 요청 - fileName과 contentType을 JSON으로 보낸다

Presigned URL 응답 - presignedUrl과 key가 반환된다

응답의 `presignedUrl` 이 출입증입니다. 이 URL로 S3에 직접 파일을 올릴 수 있습니다. `key` 는 S3 내부에서의 저장 경로입니다.

> 체크포인트 1 - Presigned URL 발급 확인
>
> 응답 JSON에 `presignedUrl` 과 `key` 두 필드가 있는지 확인합니다. `presignedUrl` 은 `https://버킷이름.s3.ap-northeast-2.amazonaws.com/original/...` 형태의 긴 URL이고 끝에 `X-Amz-Signature=...` 파라미터가 붙어 있습니다. 이 형태가 보이면 성공입니다.

발급받은 Presigned URL로 S3에 직접 이미지를 업로드합니다. Postman에서 PUT 요청을 보냅니다.

S3 PUT 헤더 - Content-Type을 presigned 요청과 동일하게 맞춘다

S3 PUT Body - Binary 탭에서 이미지 파일을 직접 선택한다

S3 업로드 성공 - 200 OK가 반환되면 S3에 파일이 올라간 것이다

S3 확인 - original 폴더에 업로드한 이미지가 저장되어 있다

원본 이미지 확인 - S3에 저장된 이미지를 미리보기로 확인할 수 있다

서버는 출입증만 발급했을 뿐입니다. 파일은 클라이언트가 S3에 직접 올렸습니다. 4장에서 서버가 Base64를 디코딩하고 파일을 쓰던 부하가 사라졌습니다.

> 체크포인트 2 - S3 업로드 확인

> Postman에서 PUT 요청 후 200 OK가 반환되었는지 확인합니다. AWS S3 콘솔에서 `original/` 폴더를 열어 방금 업로드한 파일이 보이면 성공입니다. 파일을 클릭하고 열기 버튼을 누르면 업로드한 이미지를 미리볼 수 있습니다.

5.4 Lambda 리사이즈 구현

S3에 이미지가 올라오면 자동으로 리사이즈하는 Lambda 함수를 만듭니다. Lambda는 AWS에서 제공하는 서버리스 함수입니다. 서버를 따로 띄울 필요 없이 이벤트가 발생하면 코드가 실행됩니다. 외부 창고에 상주하는 직원처럼 "물건이 들어오면 축소본 만들기"를 자동으로 수행합니다.

5.2절에서 생성한 Lambda 함수의 코드 탭으로 이동합니다. 코드 소스 에디터에 아래 코드를 입력합니다.

```
bucket = event["Records"][0]["s3"]["bucket"]["name"]
key = event["Records"][0]["s3"]["object"]["key"]

original_obj = s3.get_object(Bucket=bucket, Key=key)
image = Image.open(io.BytesIO(original_obj["Body"].read()))
image = image.convert("RGB")
image.thumbnail((800, 800))

buffer = io.BytesIO()
image.save(buffer, format="JPEG", quality=85)

uuid = key.split("/")[-1].split(".")[0]
resized_key = f"resized/{uuid}.jpg"
s3.put_object(Bucket=bucket, Key=resized_key,
              Body=buffer.getvalue(), ContentType="image/jpeg")
```

`event` 에서 버킷 이름과 파일 경로를 꺼냅니다. S3에서 원본 이미지를 내려받아 Pillow 라이브러리로 열고 800x800 이내로 리사이즈합니다. JPEG로 변환해서 `resized/` 폴더에 저장합니다. 원본이 `original/abc.png` 였다면 리사이즈 결과는 `resized/abc.jpg` 가 됩니다.

Pillow는 Lambda에 기본 내장되어 있지 않습니다. 5.2절에서 Layer를 이미 추가했으므로 코드만 작성하면 됩니다.

코드 입력 후 왼쪽 상단의 Deploy 버튼을 클릭합니다. "Successfully updated the function" 메시지가 나타나면 배포 완료입니다.

Lambda 배포 - 코드를 작성하고 Deploy 버튼으로 배포한다

> 체크포인트 3 - Lambda 트리거 확인
>
> 5.3절에서 S3에 업로드한 이미지가 있다면 AWS S3 콘솔에서 `resized/` 폴더를 확인합니다. `original/` 에 올린 파일과 같은 UUID의 `.jpg` 파일이 `resized/` 에 생성되어 있으면 Lambda가 정상 동작하는 것입니다. 파일이 없다면 3~5초 정도 기다린 뒤 새로고침합니다. Lambda 콘솔의 모니터링 탭에서 최근 호출 로그를 확인할 수도 있습니다.

리사이즈 확인 - resized 폴더에 축소된 이미지가 자동으로 생성되었다

5.5 Spring 서버 구현 2 - Resized URL 저장

클라이언트가 S3에 업로드를 완료한 뒤 Spring 서버에 "업로드 끝났어"라고 알려주는 `/complete` 엔드포인트를 만듭니다. 외부 창고에 물건을 넣은 뒤 집주인에게 보관 위치를 알려주는 단계입니다.

아래 코드를 `ImageService.java` 에 작성합니다.

```
String uuid = originalKey
        .replace("original/", "").split("\\.")[0];
String resizedKey = "resized/" + uuid + ".jpg";
String base = "https://" + bucket + ".s3."
        + region + ".amazonaws.com/";
String originalUrl = base + originalKey;
String resizedUrl = base + resizedKey;

imageRepository.save(ImageEntity.builder()
        .originalUrl(originalUrl)
        .resizedUrl(resizedUrl)
        .createdAt(LocalDateTime.now()).build());
```

클라이언트가 보내온 `originalKey` 에서 UUID를 추출합니다. `original/abc-123.png` 에서 `abc-123` 을 꺼내서 `resized/abc-123.jpg` 를 조합합니다. 원본 URL과 리사이즈 URL을 모두 DB에 저장합니다. 목록 화면에서는 리사이즈 URL을 보여주고 상세 화면에서는 원본 URL을 보여주면 됩니다.

컨트롤러입니다. 아래 코드를 `ImageController.java` 에 작성합니다.

```
@PostMapping("/complete")
public ImageResponse.DTO completeUpload(
        @RequestBody ImageRequest.CompleteDTO reqDTO) {
    return imageService.checkAndSave(reqDTO);
}
```

Postman으로 /complete를 호출합니다.

/complete 요청 - originalKey를 JSON으로 보낸다

/complete 응답 - originalUrl과 resizedUrl이 모두 저장되어 반환된다

DB 확인 - H2 콘솔에서 원본 URL과 리사이즈 URL이 저장된 것을 확인한다

> 체크포인트 4 - /complete API + DB 저장 확인
>
> `/complete` 응답에 `originalUrl` 과 `resizedUrl` 이 모두 포함되어 있는지 확인합니다. `http://localhost:8080/h2-console` 에 접속해서 `SELECT * FROM IMAGE_TB` 를 실행합니다. 방금 저장한 데이터가 조회되면 성공입니다. `resizedUrl` 을 브라우저 주소창에 붙여넣으면 리사이즈된 이미지를 직접 볼 수 있습니다.

5.6 전체 흐름 통합

목록 조회와 상세 조회 API입니다.

```
@GetMapping("/list")
public List<ImageResponse.DTO> getAllImages() {
    return imageService.listAll();
}

@GetMapping("/{id}")
public ImageResponse.DTO getImageDetail(
        @PathVariable Long id) {
    return imageService.findById(id);
}
```

3단계 통합 실습입니다. Postman에서 순서대로 실행합니다.

순서	요청	설명
1	POST /presigned	Presigned URL 발급
2	PUT {presignedUrl}	S3에 이미지 직접 업로드
3	POST /complete	업로드 완료 알림 + DB 저장
4	GET /list	저장된 이미지 목록 확인
5	GET /{id}	특정 이미지 상세 확인

목록 조회 - 저장된 모든 이미지의 원본 URL과 리사이즈 URL이 반환된다

상세 조회 - 특정 이미지의 전체 정보가 반환된다

비유	기술 용어	정식 정의
외부 창고	S3 (Simple Storage Service)	AWS의 객체 스토리지 서비스. 파일을 버킷 단위로 저장하고 HTTP로 접근한다
출입증	Presigned URL	임시 서명이 포함된 URL. 인증 없이도 지정된 시간 동안 S3에 업로드/다운로드할 수 있다
창고 직원	Lambda	AWS의 서버리스 컴퓨팅 서비스. 이벤트가 발생하면 코드가 자동 실행된다
물건 도착 알림	S3 이벤트 트리거	S3에 객체가 생성/삭제될 때 지정된 서비스(Lambda 등)를 자동 호출하는 기능
축소본	이미지 리사이즈	원본 이미지의 해상도를 줄여 용량을 낮춘 버전을 생성하는 처리
보관 위치 기록	DB 메타데이터 저장	파일의 URL, 생성일 등 부가 정보를 데이터베이스에 기록하는 과정

이것만은 기억하자

서버가 모든 일을 직접 할 필요는 없습니다. Presigned URL을 쓰면 서버는 허가증만 발급하고 클라이언트가 S3에 직접 파일을 올립니다. 서버는 파일을 한 번도 만지지 않으므로 디스크 부담도 네트워크 부담도 없습니다. Lambda를 연결하면 이미지가 올라올 때마다 리사이즈 같은 후처리가 자동으로 실행됩니다. 서버가 해야 할 일은 URL 발급과 결과 기록, 이 두 가지뿐입니다.

다음 장에서는 사용자에게 실시간으로 알림을 보내야 하는 상황을 만납니다.

6장. 새 댓글이 달렸는데 왜 새로고침을 해야 하죠

알림이 안 와요

금요일 오후, 팀 내부 게시판 기능을 막 배포한 날이었습니다.

팀장: “게시판 댓글 알림 기능 좀 넣어줘. 댓글 달리면 바로 알림이 떠야 해.”

오픈이: “알림이요? 지금도 새로고침하면 보이긴 하는데…”

팀장: “새로고침을 누가 계속 해? 카카오톡처럼 바로 떠야지.”

(카카오톡처럼?)

카카오톡에서 메시지가 오면 화면을 새로고침하지 않습니다. 그냥 뜹니다. 지금 만든 게시판은 댓글이 달려도 사용자가 직접 새로고침을 눌러야 보입니다. “실시간”이라는 단어가 머릿속에서 맴돌았지만 어디서부터 시작해야 할지 감이 오지 않았습니다.

선배 에게 물어봤습니다.

오픈이: “실시간 알림을 만들어야 하는데 어떻게 해?”

선배: “방법이 세 가지 있어. 쉬운 것부터 말해줄게.”

선배 가 화이트보드에 그림을 그리기 시작했습니다.

식당에 주문한 음식이 나왔는지 확인하는 상황을 떠올려 봅니다.

첫 번째 방법은 카운터에 직접 가서 물어보는 것입니다. “제 음식 나왔어요?” 아직이면 자리로 돌아갔다가 2분 뒤에 다시 가서 묻습니다. 또 아직이면 다시 돌아갑니다. 음식이 나올 때까지 이걸 반복합니다. 이것이 Polling 입니다. 단순하지만 바쁜 식당에서 손님이 열 명만 되어도 카운터 앞이 북적입니다.

두 번째 방법은 진동벨입니다. 주문할 때 진동벨을 받아서 자리에 앉습니다. 음식이 나오면 벨이 울립니다. 손님이 카운터에 갈 필요가 없습니다. 가게에서 알아서 알려줍니다. 이것이 SSE(Server-Sent

Events) 입니다. 가게에서 손님 방향으로만 신호를 보냅니다. 손님이 벨로 가게에 뭔가를 보낼 수는 없습니다. 단방향입니다.

세 번째 방법은 전화입니다. 주문하면서 전화를 연결해 둡니다. 가게에서 "음식 나왔어요"라고 말하면 손님이 "네 갈게요"라고 대답합니다. 손님도 "혹시 반찬 추가할 수 있어요?"라고 물을 수 있습니다. 양쪽이 자유롭게 말합니다. 이것이 WebSocket 입니다. 양방향 통신입니다. 전화를 걸 때도 "여보세요"부터 시작하는 약속이 있듯이 WebSocket 위에도 메시지를 주고받는 약속이 있습니다. 그게 STOMP 입니다.

선배: "단순한 알림이면 진동벨이면 충분해. 근데 채팅처럼 서로 주고받아야 하면 전화가 필요하고."

오픈이: "세 가지를 다 만들어 보면 감이 오겠다."

선배: "이건 세 가지 방식을 각각 따로 만들어 볼 거야. 같은 채팅 기능인데 구현이 전부 달라."

(같은 기능을 다르게 만들어 보면 차이가 바로 보이겠지.)

이제 직접 만들어 보겠습니다.

이 장의 실습 코드는 아래 세 개 레포에서 확인할 수 있습니다.

```
git clone https://github.com/metacoding-11-spring-reference/spring-polling
git clone https://github.com/metacoding-11-spring-reference/spring-sse
git clone https://github.com/metacoding-11-spring-reference/spring-websoket
```

```
spring-polling/
├── ChatController.java          [설명] 알림 CRUD API
├── ChatService.java             [설명] 인메모리 알림 저장
└── templates/index.mustache     [설명] Ajax setInterval 폴링

spring-sse/
├── ChatController.java          [실습] SseEmitter 구독/발행
├── SseEmitters.java             [실습] Emitter 관리
└── templates/index.mustache     [설명] EventSource 클라이언트

spring-websoket/
├── WebSocketConfig.java         [설명] STOMP 설정
├── ChatController.java          [설명] @MessageMapping 핸들러
└── templates/index.mustache     [설명] SockJS + STOMP 클라이언트
```

그림 6-1: 이번 챕터의 실습 흐름

6.1 실시간 통신이란

웹에서 “실시간”을 구현하는 방식은 크게 세 가지입니다.

구분	Polling	SSE	WebSocket
통신 방향	요청/응답 반복	서버 -> 클라이언트(단방향)	양방향
연결	매 요청마다	1회 연결 유지	1회 연결 유지
지연	폴링 주기만큼	거의 실시간	실시간
구현 난이도	낮음	중간	높음
적합 케이스	상태 확인	알림/진행률	채팅/협업

식당 비유로 정리하면 Polling은 카운터에 직접 물어보기, SSE는 진동벨, WebSocket은 전화입니다. 카운터에 물어보는 건 손님이 움직여야 하고 진동벨은 가게만 신호를 보내고 전화는 양쪽이 자유롭게 대화합니다.

그림 6-1: 세 방식의 통신 방향 비교

6.2 Polling 구현 (Ajax)

가장 단순한 방식부터 시작합니다. 서버에 채팅 저장과 조회 API를 만들고 클라이언트가 2초마다 조회 API를 호출합니다.

서버의 채팅 저장과 조회 API입니다.

```
@PostMapping("/chats")
@ResponseBody
public ResponseEntity<Chat> save(@RequestBody ChatRequest req) {
    Chat saved = chatService.save(req);
    return ResponseEntity.ok(saved);
}

@GetMapping("/chats")
@ResponseBody
public ResponseEntity<List<Chat>> list() {
    return ResponseEntity.ok(chatService.findAll());
}
```

`save()` 는 채팅 메시지를 저장하고 `list()` 는 전체 메시지를 반환합니다. 일반적인 CRUD API와 다를 것이 없습니다.

클라이언트의 폴링 코드입니다.

```
async function loadMessages() {
  const res = await fetch("/chats");
  const data = await res.json();
  const box = document.getElementById("chat-box");
  box.innerHTML = "";
  data.forEach(chat => {
    const li = document.createElement("li");
    li.innerText = chat.message;
    box.appendChild(li);
  });
}
loadMessages();
setInterval(loadMessages, 2000);
```

`setInterval(loadMessages, 2000)` 이 핵심입니다. 2초마다 `loadMessages()` 를 호출해서 서버에 "새 메시지 있어?"라고 물어봅니다. 새 메시지가 없어도 요청은 나갑니다. 카운터에 가서 "아직이요?"라고 물어보는 것과 같습니다.

DevTools의 Network 탭을 열면 2초 간격으로 요청이 반복되는 것을 확인할 수 있습니다.

그림 6-2: Polling DevTools - 2초마다 /chats 요청이 반복된다. 새 메시지가 없어도 요청은 계속 나간다

Network 탭에서 `/chats` 요청이 2초 간격으로 반복해서 나타나면 성공입니다. 채팅을 입력하지 않았는데도 요청이 계속 쌓이는 것을 확인해 보세요. Polling의 특성이 그대로 드러납니다.

동작은 합니다. 하지만 문제가 보입니다. 새 메시지가 없는데도 매번 요청을 보냅니다. 사용자가 100명이면 2초마다 100개의 요청이 서버에 쏟아집니다. 단순하지만 비효율적입니다.

6.3 SSE 구현 (Server-Sent Events)

SSE는 클라이언트가 한 번 연결하면 서버가 필요할 때마다 데이터를 밀어줍니다. 진동벨을 받아 놓으면 가게에서 알아서 울려주는 것과 같습니다.

먼저 Emitter를 관리하는 클래스입니다. 아래 코드를 `SseEmitters.java` 에 작성합니다.

```
private final Map<String, SseEmitter> emitters
    = new ConcurrentHashMap<>();

public void add(String clientId, SseEmitter emitter) {
```

```
        emitters.put(clientId, emitter);
        emitter.onCompletion(() -> emitters.remove(clientId));
        emitter.onTimeout(() -> emitters.remove(clientId));
    }

    public void sendAll(Chat chat) {
        emitters.forEach((id, emitter) -> {
            emitter.send(SseEmitter.event()
                .name("chat").data(chat));
        });
    }
```

`ConcurrentHashMap` 으로 연결된 클라이언트들의 Emitter를 관리합니다. `add()` 는 새 클라이언트가 연결될 때 Emitter를 저장하고 연결이 끊기면 자동으로 제거합니다. `sendAll()` 은 새 채팅이 등록되면 연결된 모든 클라이언트에게 메시지를 보냅니다. 실제로는 연결이 끊긴 Emitter에 전송하면 예외가 발생하므로 try-catch로 감싸야 합니다.

SSE 연결 엔드포인트입니다. 아래 코드를 `ChatController.java` 에 작성합니다.

```
    @GetMapping(value = "/chats/connect",
        produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public ResponseEntity<SseEmitter> connect() {
        String clientId = session.getId();
        SseEmitter emitter = new SseEmitter(60 * 1000L);
        sseEmitters.add(clientId, emitter);
        emitter.send(SseEmitter.event()
            .name("connect").data("connected"));
        return ResponseEntity.ok(emitter);
    }
```

`produces = MediaType.TEXT_EVENT_STREAM_VALUE` 가 이 엔드포인트를 일반 API가 아닌 SSE 스트림으로 만듭니다. `SseEmitter` 를 생성하면서 타임아웃을 60초로 설정합니다. 연결 즉시 `connect` 이벤트를 보내서 클라이언트에게 "연결됐어"라고 알려줍니다.

채팅 저장 시 모든 클라이언트에게 알립니다.

```
    @PostMapping("/chats")
    public ResponseEntity<Chat> save(@RequestBody ChatRequest req) {
        Chat saved = chatService.save(req);
```

```
    sseEmitters.sendAll(saved);
    return ResponseEntity.ok(saved);
}
```

`chatService.save()` 로 저장한 뒤 `sseEmitters.sendAll()` 로 연결된 모든 클라이언트에게 새 메시지를 보냅니다. Polling과 달리 클라이언트가 물어볼 필요가 없습니다. 서버가 알아서 알려줍니다.

클라이언트의 SSE 수신 코드입니다.

```
const sse = new EventSource("/chats/connect");
sse.addEventListener("chat", (e) => {
  const chat = JSON.parse(e.data);
  appendChat(chat.message);
});
```

`EventSource` 가 SSE 연결을 담당합니다. 한 번 연결하면 서버에서 `chat` 이벤트가 올 때마다 자동으로 콜백이 실행됩니다. Polling의 `setInterval` 이 사라졌습니다.

DevTools에서 SSE 연결을 확인합니다.

그림 6-3: SSE 연결 - /chats/connect 요청이 한 번만 나가고 연결이 유지된다

그림 6-4: connect 이벤트 - 연결 즉시 서버가 “connected” 메시지를 보내온다

Network 탭에서 `/chats/connect` 요청의 Type이 `eventsource` 로 표시되고 EventStream 탭에 `connect` 이벤트가 보이면 SSE 연결이 성공한 것입니다.

이제 다른 브라우저 탭을 열고 채팅을 보내 봅니다.

그림 6-5: chat 이벤트 - 다른 사용자가 채팅을 보내면 서버가 자동으로 밀어준다

그림 6-6: 실시간 동기화 - 두 브라우저에서 새로고침 없이 채팅이 바로 표시된다

한쪽 브라우저에서 채팅을 입력했을 때 다른 쪽 브라우저에 새로고침 없이 메시지가 나타나면 성공입니다. Network 탭을 Polling 때와 비교해 보세요. 2초 간격 요청이 사라지고 하나의 연결만 유지되는 것을 확인할 수 있습니다.

Polling에서 2초마다 반복되던 요청이 사라졌습니다. 연결은 한 번이고 서버가 필요할 때만 데이터를 보냅니다. 알림, 진행률 표시처럼 서버에서 클라이언트 방향으로만 데이터를 보내는 경우에 적합합니다.

6.4 WebSocket 구현 (STOMP)

WebSocket은 클라이언트와 서버가 양방향으로 자유롭게 메시지를 주고받습니다. 전화처럼 양쪽이 동시에 말할 수 있습니다. Spring에서는 STOMP(Simple Text Oriented Messaging Protocol) 를 함께 사용하면 메시지 라우팅이 편해집니다.

WebSocket 설정 클래스입니다.

```
@Override
public void registerStompEndpoints(StompEndpointRegistry registry) {
    registry.addEndpoint("/ws")
        .setAllowedOriginPatterns("*");
}

@Override
public void configureMessageBroker(MessageBrokerRegistry registry) {
    registry.setApplicationDestinationPrefixes("/app");
    registry.enableSimpleBroker("/topic");
}
```

`addEndpoint("/ws")` 로 WebSocket 연결 주소를 `/ws` 로 지정합니다. `setApplicationDestinationPrefixes("/app")` 은 클라이언트가 서버에 메시지를 보낼 때의 접두사입니다. `enableSimpleBroker("/topic")` 은 서버가 클라이언트에게 메시지를 보낼 때의 접두사입니다. `/app` 으로 보내면 서버가 처리하고 `/topic` 으로 구독하면 서버가 보내주는 메시지를 받습니다.

메시지 핸들러입니다.

```
@MessageMapping("/chats")
public void handle(ChatRequest payload) {
    Chat saved = chatService.save(payload);
    messagingTemplate.convertAndSend("/topic/chats", saved);
}
```

`@MessageMapping("/chats")` 는 클라이언트가 `/app/chats` 로 보낸 메시지를 이 메서드가 처리한다는 뜻입니다. 저장 후 `messagingTemplate.convertAndSend()` 로 `/topic/chats` 를 구독한 모든 클라이언트에게 메시지를 보냅니다.

클라이언트의 WebSocket 연결과 구독 코드입니다.

```
const stompClient = Stomp.over(new WebSocket(socketUrl));
stompClient.connect({}, () => {
  stompClient.subscribe("/topic/chats", (msg) => {
    const data = JSON.parse(msg.body);
    appendChat(data.message);
  });
});
// 전송
stompClient.send("/app/chats", {},
    JSON.stringify({ message }));
```

`Stomp.over()` 로 WebSocket 위에 STOMP 프로토콜을 올립니다. `subscribe("/topic/chats")` 로 서버의 메시지를 받고 `send("/app/chats")` 로 서버에 메시지를 보냅니다. 양방향 통신이 한 연결 안에서 이루어집니다.

DevTools에서 WebSocket 연결을 확인합니다.

그림 6-7: WebSocket 연결 - /ws 엔드포인트로 한 번 연결되면 끊기지 않는다

그림 6-8: STOMP 구독 - /topic/chats를 구독하면 서버가 보내는 메시지를 받을 준비가 된다

Network 탭에서 `/ws` 요청의 Status가 `101 Switching Protocols` 이고 Messages 탭에 `CONNECTED` 프레임이 보이면 WebSocket + STOMP 연결이 성공한 것입니다.

채팅을 입력해서 양방향 통신을 확인합니다.

그림 6-9: 양방향 통신 - 클라이언트가 보낸 메시지가 구독자 전원에게 전달된다

Messages 탭에서 보낸 메시지(화살표 위)와 받은 메시지(화살표 아래)가 모두 표시되면 성공입니다. SSE와 달리 클라이언트도 서버에 메시지를 보낼 수 있습니다. 채팅, 실시간 협업 편집, 게임처럼 양쪽이 수시로 데이터를 주고받아야 하는 경우에 적합합니다.

6.5 세 방식 비교 정리

같은 채팅 기능을 세 가지 방식으로 만들어 봤습니다. 세 프로젝트를 모두 실행한 상태에서 각각의 Network 탭을 나란히 열어 보면 차이가 한눈에 들어옵니다. Polling은 요청이 끊임없이 쌓이고 SSE는 하나의 연결에서 서버 이벤트만 내려오고 WebSocket은 하나의 연결에서 양방향 메시지가 오갑니다.

구분	Polling	SSE	WebSocket
연결 방식	매번 새 요청	한 번 연결 유지	한 번 연결 유지
통신 방향	클라이언트 -> 서버 (반복)	서버 -> 클라이언트	양방향
구현 핵심	setInterval + fetch	SseEmitter + EventSource	STOMP + WebSocket
적합 시나리오	대시보드 새로고침	알림, 진행률	채팅, 협업

비유	기술 용어	정식 정의
카운터에 물어보기	Polling	클라이언트가 일정 주기로 서버에 요청을 보내 새 데이터를 확인하는 방식
진동벨	SSE (Server-Sent Events)	서버가 클라이언트에게 단방향으로 이벤트 스트림을 보내는 HTTP 기반 기술
전화	WebSocket	하나의 TCP 연결 위에서 클라이언트와 서버가 양방향으로 메시지를 주고받는 프로토콜
전화 위의 약속	STOMP	WebSocket 위에서 메시지 라우팅(구독/발행)을 쉽게 해주는 텍스트 기반 프로토콜
벨 보관함	SseEmitter	Spring에서 SSE 연결을 관리하는 객체. 서버가 이벤트를 보내는 통로 역할을 한다
벨 수신기	EventSource	브라우저에서 SSE 서버에 연결하고 이벤트를 수신하는 JavaScript API

이것만은 기억하자

실시간 통신은 요구사항에 따라 방식을 고릅니다. 새 데이터를 확인만 하면 되는 곳에는 Polling이 가장 빠르게 구현됩니다. 서버에서 클라이언트로 알림을 보내야 하면 SSE가 효율적입니다. 양쪽이 자유

롭게 메시지를 주고받아야 하면 WebSocket을 씁니다. 세 방식의 코드를 나란히 비교해 보면 차이는 연결 방식과 데이터 전달 방향에 있습니다. 기능이 같아도 “누가 먼저 말하느냐”에 따라 구현이 달라집니다.

다음 장에서는 영상 콘텐츠 서비스를 준비합니다.

7장. 영상도 WebSocket으로 되지 않아?

1GB짜리 영상

사내 교육 플랫폼에 영상 강의 기능을 넣어 달라는 요청이 들어왔습니다.

오픈이: “영상이면 지난번에 만든 WebSocket으로 스트리밍하면 되지 않나요?”

팀장: “WebSocket은 채팅용이야. 영상은 차원이 다른 문제야.”

(차원이 다르다고?)

직접 해봤습니다. 1GB짜리 교육 영상을 서버에 올리고 브라우저에서 재생 버튼을 눌렀습니다. 로딩 스피너가 돌기 시작했습니다. 30초가 지나도 재생이 시작되지 않았습니다. 1분이 지나자 브라우저 탭이 멈췄습니다. 파일 전체를 다운로드해야 재생이 시작되는 구조였습니다.

선배: “그거 통째로 보내면 안 돼. 잘게 쪼개서 보내야 해.”

오픈이: “쪼갠다고?”

선배: “유튜브 생각해 봐. 영상 틀면 바로 재생되잖아. 전체를 다 받아서 트는 게 아니야.”

택배를 보낼 때를 떠올려 봅니다.

작은 상자 하나는 일반 택배로 보내면 됩니다. 집 앞에서 받아서 바로 열어봅니다. 그런데 이삿짐처럼 짐이 트럭 한 대 분량이라면 이야기가 달라집니다. 트럭이 도착할 때까지 아무것도 못 합니다. 짐을 전부 내려야 원하는 물건을 꺼낼 수 있습니다.

이삿짐 업체는 다른 방식을 씁니다. 짐을 상자 단위로 나눠서 번호를 매깁니다. 1번 상자, 2번 상자, 3번 상자. 1번 상자가 도착하면 바로 풀기 시작합니다. 2번 상자가 오는 동안 1번 상자의 짐을 정리합니다. 전체가 도착하지 않아도 하나씩 처리할 수 있습니다.

영상 스트리밍도 같은 원리입니다. 1GB 영상을 통째로 보내면 브라우저는 전부 받을 때까지 기다립니다. 하지만 10초짜리 조각으로 잘라서 보내면 첫 번째 조각이 도착하는 순간 재생이 시작됩니다. 나머지 조각은 재생하는 동안 뒤에서 도착합니다.

상자에 번호를 매기듯 조각 파일에도 순서 목록이 필요합니다. "1번 다음은 2번, 그 다음은 3번"이라는 재생 목록이 있어야 브라우저가 순서대로 요청합니다. 이 목록이 m3u8 파일이고 조각 하나하나가 ts 파일입니다. HTTP 위에서 영상을 실시간으로 잘라 보내는 이 방식 전체를 HLS(HTTP Live Streaming)라고 부릅니다.

선배: "이삿짐을 상자로 나눠서 번호 매기는 거랑 똑같아. 목록표 한 장이랑 상자 여러 개."

(목록표가 m3u8이고 상자가 ts 파일이구나.)

오픈이: "그러면 영상을 쪼개는 건 누가 해?"

선배: "FFmpeg이라는 도구가 해. 영상을 넣으면 알아서 잘라줘."

이제 직접 만들어 보겠습니다.

이 장의 실습 코드는 아래 레포에서 확인할 수 있습니다.

```
git clone https://github.com/metacoding-11-spring-reference/spring-hls
```

```
spring-hls/
├── HlsController.java         [설명] 업로드 + 스트리밍 엔드포인트
├── HlsService.java            [설명] FFmpeg 인코딩 + 파일 로딩
├── HlsApplication.java        [참고] @EnableAsync 활성화
├── application.properties     [참고] 업로드 용량 설정 (2GB)
└── templates/index.mustache   [참고] HLS.js 플레이어 UI
```

이번 챕터의 실습 흐름

환경 준비

HLS 인코딩에는 FFmpeg 이 필요합니다. 운영체제별 설치 방법입니다.

macOS

```
brew install ffmpeg
```

Windows

```
choco install ffmpeg
```

Chocolatey가 없다면 FFmpeg 공식 사이트에서 바이너리를 받아 시스템 PATH에 추가합니다.

Linux (Ubuntu/Debian)

```
sudo apt install ffmpeg
```

설치가 끝나면 버전을 확인합니다.

```
ffmpeg -version
```

`ffmpeg version` 으로 시작하는 출력이 나오면 성공입니다.

[CAPTURE NEEDED: ffmpeg -version 실행 결과 - 버전 정보와 빌드 옵션이 출력되는 터미널 화면]

ffmpeg -version 실행 결과

테스트에 사용할 영상 파일도 준비합니다. 1분 이상 길이의 MP4 파일을 권장합니다. 너무 짧은 영상은 ts 조각이 하나만 생성되어 스트리밍 효과를 체감하기 어렵습니다.

7.1 HLS 스트리밍이란

영상을 브라우저에서 바로 재생하려면 파일을 잘게 쪼개서 순서대로 보내야 합니다. HLS는 이 과정을 HTTP 위에서 처리하는 표준 방식입니다.

구조는 두 가지 파일로 이루어집니다. m3u8 은 재생 목록입니다. 어떤 조각을 어떤 순서로 재생할지 적혀 있습니다. ts 는 실제 영상 조각입니다. 보통 10초 단위로 잘립니다. 브라우저는 m3u8 파일을 먼저 받아서 목록을 확인하고 ts 파일을 순서대로 요청합니다.

전체 흐름은 이렇습니다. 사용자가 영상을 업로드하면 서버가 FFmpeg으로 원본을 ts 조각으로 변환합니다. 변환이 끝나면 m3u8 목록 파일이 생성됩니다. 브라우저는 m3u8을 요청해서 목록을 받고 ts 조각을 하나씩 가져와 재생합니다.

업로드부터 브라우저 재생까지의 HLS 흐름

7.2 업로드 API 구현

영상 파일을 받아서 서버에 저장하고 HLS 변환을 시작하는 엔드포인트입니다.

```
@PostMapping("/upload")
public ResponseEntity<String> uploadVideo(
```

```
        @RequestParam("file") MultipartFile file) throws IOException {
    String savedName = hlsService.saveOriginalVideo(file);
    hlsService.convertToHls(savedName);
    return ResponseEntity.ok("HLS 변환 완료: " + savedName);
}
```

`MultipartFile` 로 영상을 받아서 `saveOriginalVideo()` 로 서버에 저장합니다. 저장이 끝나면 `convertToHls()` 로 HLS 변환을 시작합니다. `convertToHls()` 는 `@Async` 가 붙어 있어서 변환이 끝나기 전에 응답이 먼저 나갑니다.

저장 로직입니다.

```
public String saveOriginalVideo(MultipartFile file) throws IOException {
    new File(ORIGINAL_DIR).mkdirs();
    String fileName = "video.mp4";
    File saveFile = new File(ORIGINAL_DIR + fileName);
    file.transferTo(saveFile);
    return fileName;
}
```

`ORIGINAL_DIR` 디렉토리가 없으면 생성하고 업로드된 파일을 `video.mp4` 로 저장합니다. `transferTo()` 가 실제 파일 쓰기를 담당합니다.

Postman으로 영상을 업로드한 결과입니다.

영상 업로드 - Postman으로 파일을 보내면 HLS 변환이 시작된다

확인 포인트 - 서버 콘솔에 FFmpeg 인코딩 시작 로그가 출력됩니다. `convertToHls()` 가 `@Async` 로 실행되므로 Postman 응답이 먼저 돌아오고 서버 로그에 인코딩 진행 상황이 이어서 나타나면 성공입니다.

7.3 FFmpeg 인코딩과 HLS 조각화

업로드된 영상을 FFmpeg으로 쪼개는 과정입니다. 720p와 1080p 두 화질로 변환합니다.

```
FFmpegBuilder builder720 = new FFmpegBuilder()
    .setInput(inputPath)
    .addOutput(output720)
    .addExtraArgs("-b:v", "2500k")
    .addExtraArgs("-maxrate", "2500k")
```

```
    .addExtraArgs("-bufsize", "5000k")
    .addExtraArgs("-vf", "scale=-2:720")
    .addExtraArgs("-hls_time", "10")
    .addExtraArgs("-hls_list_size", "0")
    .addExtraArgs("-f", "hls")
    .done();
```

각 옵션의 역할입니다.

옵션	역할
`-vf scale=-2:720`	세로 720px, 가로는 비율에 맞게 자동 계산
`-b:v 2500k`	비트레이트 2500kbps
`-hls_time 10`	10초 단위로 영상을 조각냄
`-hls_list_size 0`	모든 조각을 목록에 포함
`-f hls`	출력 포맷을 HLS로 지정

`FFmpegBuilder` 는 Java에서 FFmpeg 명령을 만들어주는 래퍼입니다. `setInput()` 으로 원본 영상 경로를 넣고 `addOutput()` 으로 출력 경로를 지정합니다. `-hls_time 10` 이 영상을 10초 단위로 자르는 핵심 옵션입니다. 같은 방식으로 1080p 빌더도 만들어서 두 화질을 동시에 생성합니다.

변환이 끝나면 서버에 파일이 생성됩니다.

HLS 파일 생성 - m3u8 목록 파일과 ts 조각 파일이 만들어졌다

확인 포인트 - 인코딩이 끝난 뒤 출력 디렉토리에서 `ls` 를 실행합니다. `video.m3u8` 파일 1개와 `video0.ts` , `video1.ts` … 형태의 ts 파일 여러 개가 보이면 성공입니다. 720p 폴더와 1080p 폴더에 각각 파일이 생성되어야 합니다.

```
ls hls-videos/720p/
ls hls-videos/1080p/
```

[CAPTURE NEEDED: 720p, 1080p 디렉토리에서 ls 실행 결과 - m3u8 파일과 ts 조각 파일 목록이 출력되는 터미널 화면]

인코딩 완료 후 생성된 파일 목록

이삿짐 비유로 보면 FFmpeg이 짐을 상자에 나눠 담는 작업자입니다. 상자 크기를 10초로 정하고 (`-hls_time 10`) 번호를 매겨서(`-hls_list_size 0`) 목록표(m3u8)와 함께 정리합니다.

7.4 HLS 스트리밍 제공

조각으로 나눈 파일을 브라우저에 전달하는 엔드포인트입니다.

```
@GetMapping("/hls/{quality}/{fileName}.m3u8")
public ResponseEntity<Resource> getHlsPlaylist(
        @PathVariable String quality,
        @PathVariable String fileName) throws IOException {
    Resource resource = hlsService.loadHlsFile(
        quality, fileName + ".m3u8");
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.parseMediaType(
        "application/vnd.apple.mpegurl"));
    return new ResponseEntity<>(resource, headers, HttpStatus.OK);
}
```

`{quality}` 는 `720p` 또는 `1080p` 입니다. `Content-Type` 을 `application/vnd.apple.mpegurl` 로 지정해야 브라우저가 이 응답을 HLS 재생 목록으로 인식합니다. ts 파일을 제공하는 엔드포인트도 같은 구조입니다. 경로만 `.ts` 로 바뀌고 `Content-Type` 이 `video/mp2t` 로 달라집니다.

브라우저에서 재생하려면 HLS.js 라이브러리가 필요합니다.

```
var url = `http://localhost:8080/hls/${quality}/video.m3u8`;
if (Hls.isSupported()) {
    var hls = new Hls();
    hls.loadSource(url);
    hls.attachMedia(video);
}
```

`Hls.isSupported()` 로 브라우저가 HLS를 지원하는지 확인합니다. `loadSource()` 에 m3u8 주소를 넣으면 HLS.js가 목록을 파싱하고 ts 파일을 순서대로 요청합니다. `attachMedia()` 로 비디오 태그에 연결하면 재생이 시작됩니다.

Postman으로 m3u8 엔드포인트를 확인합니다.

m3u8 엔드포인트 확인 - 재생 목록이 정상적으로 반환된다

720p와 1080p의 m3u8 응답입니다.

720p m3u8 응답 - ts 조각 파일 목록과 재생 시간이 기록되어 있다

1080p m3u8 응답 - 같은 구조지만 해상도가 다르다

브라우저에서 실제로 재생하면서 DevTools Network 탭을 열어 봅니다.

브라우저 재생 - ts 조각이 순차적으로 요청되며 영상이 끊기지 않고 재생된다

DevTools를 보면 m3u8을 한 번 받은 뒤 ts 파일이 순서대로 요청되는 것을 확인할 수 있습니다. 목록표를 받아서 상자를 하나씩 가져오는 것과 같습니다.

확인 포인트 - 브라우저에서 영상이 끊기지 않고 재생되면 성공입니다. DevTools Network 탭에서 m3u8 요청 1개, 그 뒤를 따르는 ts 요청 여러 개가 보이면 HLS 스트리밍이 정상 동작하는 것입니다. 720p/1080p 화질 전환 버튼을 눌러 화질이 바뀌는지도 확인합니다.

7.5 스트리밍 기술 선택 가이드

HLS를 만들어 봤으니 다른 스트리밍 기술과 비교해 봅니다. 각 기술은 역할이 다릅니다.

기술	역할	지연 시간	대표 사용처
RTMP	방송 송출	수 초	OBS에서 서버로 송출
HLS	HTTP 기반 배포	수 초 ~ 수십 초	VOD, 라이브 시청
DASH	적응형 배포	수 초 ~ 수십 초	글로벌 스트리밍 서비스
WebRTC	P2P 실시간 통신	수백 ms	화상회의, 1:1 통화
RTSP	장비 스트림	수 초	CCTV, IP 카메라

선택 기준은 “지연을 얼마나 허용하느냐”입니다.

수 초의 지연이 괜찮다면 HLS 가 가장 현실적입니다. HTTP 기반이라 CDN 캐싱이 가능하고 브라우저 호환성이 좋습니다. 유튜브, 넷플릭스 같은 VOD 서비스가 HLS를 씁니다.

1초 이하의 초저지연이 필요하면 WebRTC 입니다. 화상회의나 실시간 상담에서 씁니다. 6장에서 다뤘던 WebSocket이 텍스트 메시지의 양방향 통신이라면 WebRTC는 영상/음성의 P2P 통신입니다.

방송 송출 쪽에서는 RTMP 가 여전히 표준입니다. OBS 같은 송출 프로그램이 RTMP로 서버에 보내면 서버가 이걸 HLS로 변환해서 시청자에게 배포하는 구조가 일반적입니다.

CCTV나 IP 카메라 같은 장비 스트림은 RTSP 를 씁니다. 브라우저가 아니라 전용 플레이어에서 재생합니다.

이삿짐 비유로 정리하면 HLS는 상자를 번호 매겨서 택배로 보내는 방식이고 WebRTC는 이사 당일 직접 트럭으로 실어 나르는 방식입니다. 택배는 시간이 조금 걸리지만 안정적이고 직접 운반은 빠르지만 준비가 많이 필요합니다.

비유	기술 용어	정식 정의
이삿짐을 상자로 나누기	HLS (HTTP Live Streaming)	HTTP 위에서 영상을 작은 조각(ts)으로 나누고 재생 목록(m3u8)을 통해 순차 전송하는 스트리밍 프로토콜
목록표	m3u8	HLS에서 ts 조각 파일의 순서와 재생 시간을 기록한 텍스트 기반 재생 목록 파일
번호 매긴 상자	ts (Transport Stream)	영상을 일정 시간 단위로 잘라낸 조각 파일. 보통 10초 단위
짐을 상자에 나눠 담는 작업자	FFmpeg	영상/음성 변환, 인코딩, 포맷 전환을 처리하는 오픈소스 멀티미디어 프레임워크
브라우저의 목록표 해석기	HLS.js	브라우저에서 m3u8 파일을 파싱하고 ts 조각을 순차 요청해 재생하는 JavaScript 라이브러리
택배로 보내기 vs 직접 트럭	HLS vs WebRTC	HLS는 HTTP 기반 순차 전송(지연 허용), WebRTC는 P2P 실시간 전송(초저지연)

이것만은 기억하자

큰 영상 파일을 통째로 보내면 브라우저는 전부 받을 때까지 멈춥니다. HLS는 영상을 10초 조각으로 잘라서 목록과 함께 보냅니다. 브라우저는 목록을 보고 조각을 하나씩 가져오면서 재생합니다. 전체가 도착하지 않아도 첫 번째 조각이 오는 순간 재생이 시작됩니다. 이삿짐을 상자로 나눠 번호를 매기는 것과 같습니다. 스트리밍 기술을 고를 때는 "지연을 얼마나 허용하느냐"가 기준입니다. 수 초가 괜찮으면 HLS, 1초 이하가 필요하면 WebRTC입니다.

다음 장에서는 "검색이 5초 넘게 걸린다"는 버그 리포트가 올라옵니다.

8장. 검색이 왜 이렇게 느려요?

5초짜리 검색

게시판에 검색 기능이 있었습니다. 제목이나 내용에 키워드를 넣으면 결과가 나오는 단순한 구조였습니다. 데이터가 천 건일 때는 아무 문제가 없었습니다.

데이터가 십만 건을 넘기자 상황이 달라졌습니다. 버그 리포트가 올라왔습니다.

팀장: "검색이 5초 넘게 걸린다는 리포트 들어왔어. 확인해 봐."

SQL을 열어 봤습니다. `WHERE title LIKE '%갤럭시%'` 가 전부였습니다.

(이게 왜 느린 거지?)

선배: "LIKE에 앞뒤로 % 붙이면 인덱스를 못 타. 테이블 전체를 처음부터 끝까지 훑어야 해."

오픈이: "그러면 인덱스를 걸면 되지 않아?"

선배: "앞에 %가 붙으면 인덱스가 소용없어. 데이터베이스 인덱스는 앞글자 기준으로 정렬해 놓은 거거든. '갤럭시'로 시작하는 건 찾을 수 있어도 중간에 '갤럭시'가 들어간 건 못 찾아."

(그러면 어떻게 해야 하지?)

선배: "검색 전용 엔진을 따로 두는 거야."

도서관에 갔다고 생각해 봅니다.

책이 만 권 있는 도서관에서 "스프링"이라는 단어가 들어간 책을 찾아야 합니다. 도서관에 색인 카드가 없다면 책장 첫 번째 칸부터 마지막 칸까지 책을 한 권씩 꺼내서 제목을 확인해야 합니다. 만 권을 전부 훑어야 합니다. LIKE 검색이 이 방식입니다.

색인 카드함이 있는 도서관은 다릅니다. 카드함에는 키워드별로 카드가 정리되어 있습니다. "스프링" 카드를 꺼내면 "3번 책장 위에서 두 번째, 7번 책장 아래에서 세 번째"라고 적혀 있습니다. 만 권을 훑을 필요 없이 카드 한 장만 보면 됩니다.

Elasticsearch 가 이 색인 카드함입니다. 데이터가 저장될 때마다 단어를 쪼개서 "이 단어는 몇 번 문서에 있다"는 목록을 만들어 놓습니다. 검색할 때는 이 목록만 뒤집니다. 문서 전체를 훑지 않습니다.

데이터베이스가 책장이라면 Elasticsearch는 색인 카드함입니다. 책장에서 직접 찾는 대신 카드함에서 위치를 확인하고 책장으로 가는 겁니다.

팀장: "그러면 데이터를 두 군데에 저장해야 하는 거야?"

선배: "맞아. 데이터베이스에도 넣고 검색 엔진에도 넣어. 저장은 데이터베이스가 하고 검색은 검색 엔진이 하는 거야."

(책장에 책을 꽂으면서 동시에 색인 카드도 써넣는 거구나.)

오픈이: "한번 만들어 볼게요."

이 장의 실습 코드는 아래 레포에서 확인할 수 있습니다.

```
git clone https://github.com/metacoding-11-spring-reference/docker-elasticsearch
```

```
docker-elasticsearch/
├── docker-compose.yml            [실습] ES + Kibana 구성
├── DeviceEntity.java             [설명] JPA 엔티티
├── DeviceDocument.java           [설명] ES Document 매핑
├── DeviceRepository.java         [참고] JPA Repository
├── DeviceSearchRepository.java   [참고] ES Repository
├── DeviceService.java            [실습] 이중 저장 + 검색 로직
├── DeviceController.java         [참고] 엔드포인트
├── SearchService.java            [실습] multi_match + Fuzzy 검색
└── application.properties        [실습] ES 접속 설정
```

기술 파트

이번 챕터의 실습 흐름

8.1 왜 Elasticsearch가 필요한가

LIKE 검색의 문제를 정리합니다.

`WHERE title LIKE '%갤럭시%'` 는 테이블의 모든 행을 처음부터 끝까지 읽습니다. 데이터가 천 건이면 천 번, 십만 건이면 십만 번 비교합니다. 앞에 `%` 가 붙으면 데이터베이스 인덱스(B-Tree)를 사용할 수 없습니다. B-Tree 인덱스는 값의 앞부분을 기준으로 정렬하기 때문입니다.

Elasticsearch는 역인덱스(Inverted Index) 구조를 씁니다. 문서가 저장될 때 내용을 단어 단위로 쪼개고 "이 단어가 어떤 문서에 있는지"를 기록합니다.

역인덱스 구조 - 단어 목록에서 해당 문서를 바로 찾는다

일반 인덱스가 "문서 → 단어" 방향이라면 역인덱스는 "단어 → 문서" 방향입니다. 검색할 때 단어 목록에서 해당 키워드를 찾으면 문서 번호가 바로 나옵니다. 데이터가 백만 건이어도 키워드 목록에서 한 번 찾는 것으로 끝납니다.

8.2 실습 환경 준비

Elasticsearch와 Kibana를 Docker로 띄웁니다. 아래 코드를 `docker-compose.yml` 에 작성합니다.

```yaml
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.19.8
    container_name: elasticsearch
    ports:
      - "9200:9200"
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms1g -Xmx1g
    networks:
      - es-network
  kibana:
    image: docker.elastic.co/kibana/kibana:8.19.8
    ports:
      - "5601:5601"
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    networks:
      - es-network
```

네트워크 설정과 Spring Boot 앱 컨테이너는 전체 docker-compose.yml을 참고합니다.

```bash
docker-compose up -d
```

docker-compose up - Elasticsearch, Kibana, Spring Boot 앱이 함께 올라간다

컨테이너가 올라오면 Elasticsearch 상태를 확인합니다.

```bash
curl localhost:9200
```

클러스터 이름과 버전 정보가 담긴 JSON 응답이 돌아오면 성공입니다.

localhost:9200 응답 - cluster_name, version 정보가 보이면 Elasticsearch가 정상 동작하고 있다

브라우저에서 `http://localhost:5601` 에 접속하면 Kibana 메인 화면이 나옵니다. 왼쪽 사이드바 하단의 Management 메뉴를 열고 Dev Tools 를 클릭하면 Elasticsearch에 쿼리를 직접 실행할 수 있는 콘솔이 열립니다. 8.6절에서 이 콘솔을 사용합니다.

nori 한국어 분석기 설치

Elasticsearch의 기본 분석기는 영어 기준으로 동작합니다. 한국어 형태소 분석이 필요하다면 nori 플러그인을 설치합니다. 실행 중인 ES 컨테이너 안에서 설치하는 방법입니다.

```
docker exec -it elasticsearch elasticsearch-plugin install analysis-nori
docker restart elasticsearch
```

설치 후 컨테이너를 재시작해야 적용됩니다. Dockerfile로 이미지를 만들어 두면 매번 설치할 필요가 없습니다.

```
FROM docker.elastic.co/elasticsearch/elasticsearch:8.19.8
RUN elasticsearch-plugin install analysis-nori
```

이 실습에서는 nori 없이도 동작하므로 필수는 아닙니다. 한국어 검색 품질을 높이고 싶다면 적용합니다.

Spring Boot에서 Elasticsearch에 접속하려면 `application.properties` 에 주소를 설정합니다.

```
spring.datasource.url=jdbc:h2:mem:testdb;MODE=MySQL
spring.datasource.username=sa
spring.datasource.password=
spring.elasticsearch.uris=http://elasticsearch:9200
```

H2 데이터베이스와 Elasticsearch를 동시에 사용하는 구성입니다. H2는 RDB 역할을, Elasticsearch는 검색 엔진 역할을 담당합니다.

8.3 데이터 모델 설계: RDB + ES

하나의 데이터를 두 곳에 저장하려면 모델도 두 개가 필요합니다. RDB용 엔티티와 ES용 도큐먼트입니다.

RDB용 JPA 엔티티 입니다.

```
@Entity
@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class DeviceEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String content;
}
```

JPA가 관리하는 테이블에 매핑됩니다. `id` , `title` , `content` 세 개의 컬럼을 가집니다.

ES용 도큐먼트 입니다.

```
@Document(indexName = "devices")
public class DeviceDocument {
    private Long id;
    @Field(type = FieldType.Text)
    private String title;
    @Field(type = FieldType.Text)
    private String content;
}
```

`@Document(indexName = "devices")` 가 Elasticsearch의 인덱스 이름을 지정합니다. `@Field(type = FieldType.Text)` 는 이 필드를 역인덱스 대상으로 설정합니다. Text 타입으로 지정하면 Elasticsearch가 저장 시점에 단어를 쪼개서 역인덱스를 만듭니다.

두 모델의 필드는 같지만 역할이 다릅니다. DeviceEntity는 데이터의 원본을 보관하고 DeviceDocument는 검색용 사본을 보관합니다. 도서관 비유에서 책장에 꽂힌 책이 DeviceEntity이고 색인 카드가 DeviceDocument입니다.

8.4 저장 흐름: RDB + ES 이중 저장

데이터를 저장할 때 RDB와 ES에 동시에 넣습니다. 이 방식을 이중 저장(Dual Write) 이라고 합니다.

```
@Transactional
public DeviceDocument saveDevices(DeviceDocument doc) {
```

```
    DeviceEntity entity = DeviceEntity.builder()
            .title(doc.getTitle())
            .content(doc.getContent())
            .build();
    DeviceEntity saved = deviceJpaRepository.save(entity);
    DeviceDocument savedDoc = new DeviceDocument(
            saved.getId(), saved.getTitle(), saved.getContent());
    deviceSearchRepository.save(savedDoc);
    return savedDoc;
}
```

흐름을 따라가 봅니다. 먼저 JPA Repository로 RDB에 저장합니다. RDB가 생성한 `id` 를 받아서 같은 데이터를 DeviceDocument에 담습니다. 그리고 ES Repository로 Elasticsearch에 저장합니다. `@Transactional` 이 걸려 있으므로 RDB 저장이 실패하면 롤백됩니다. 다만 Elasticsearch는 트랜잭션 범위 밖이므로 RDB 저장 성공 후 ES 저장이 실패하면 데이터 불일치가 생길 수 있습니다. 이 한계는 이벤트 기반 동기화 등으로 해결하지만 이 책의 범위를 넘어가므로 이중 저장의 특성으로 기억해 둡니다.

책장에 책을 꽂으면서 동시에 색인 카드를 써넣는 과정입니다. 카드에는 책 번호(id)를 적어 놓습니다.

Postman으로 데이터를 저장하고 결과를 확인합니다.

Postman으로 데이터 저장 - 요청이 성공하면 저장된 Document가 응답으로 온다

H2 콘솔 확인 - RDB에도 같은 데이터가 들어가 있다

중간 확인: 데이터가 제대로 들어갔는가

Kibana Dev Tools에서 인덱스를 조회합니다.

```
GET /devices/_search
{
  "query": { "match_all": {} }
}
```

`hits.total.value` 에 저장한 건수가 표시되고 `_source` 에 title, content가 보이면 성공입니다. H2 콘솔(`localhost:8080/h2-console`)에서 `SELECT * FROM DEVICE_ENTITY` 를 실행하면 RDB 쪽 데이터도 확인할 수 있습니다. 양쪽 건수가 같으면 이중 저장이 정상 동작한 것입니다.

[CAPTURE NEEDED: Kibana Dev Tools에서 devices 인덱스 match_all 검색 결과 - hits.total.value와 _source 확인]

8.5 검색 흐름: ES 검색 -> RDB 재조회

검색은 반대 방향입니다. Elasticsearch에서 키워드로 문서를 찾고 그 결과의 id로 RDB에서 원본 데이터를 가져옵니다.

```
public List<DeviceEntity> searchAll(String keyword) {
    NativeQuery query = NativeQuery.builder()
            .withQuery(q -> q.bool(b -> b
                    .should(s -> s.multiMatch(m -> m
                            .fields("title^3", "content")
                            .query(keyword)
                            .fuzziness("AUTO")))
                    .minimumShouldMatch("1")))
            .build();
    var deviceHits = operations.search(query, DeviceDocument.class);
    List<Long> ids = deviceHits.stream()
            .map(hit -> hit.getContent().getId())
            .toList();
    return deviceJpaRepository.findAllById(ids);
}
```

`multiMatch` 는 여러 필드를 동시에 검색합니다. `"title^3"` 은 제목 필드에 3배 가중치를 준다는 뜻입니다. 제목에서 일치하면 본문에서 일치하는 것보다 점수가 높습니다. 검색 결과에서 id 목록을 뽑아낸 뒤 `findAllById` 로 RDB에서 원본 엔티티를 가져옵니다.

색인 카드함에서 “갤럭시” 카드를 찾아 “3번, 7번 책장”이라는 위치를 확인한 뒤 해당 책장에서 책을 꺼내오는 것과 같습니다.

Postman 검색 - “갤럭시”로 검색하면 관련 데이터가 반환된다

중간 확인: 검색 API가 정상 동작하는가

Postman 응답에서 title 또는 content에 “갤럭시”가 포함된 항목이 돌아오면 성공입니다. 응답 JSON의 id 값이 H2 콘솔의 `DEVICE_ENTITY` 테이블 id와 일치하는지도 확인합니다. Kibana Dev Tools에서 같은 쿼리를 직접 실행해 봅니다.

```
GET /devices/_search
{
  "query": {
    "multi_match": {
      "query": "갤럭시",
      "fields": ["title^3", "content"]
```

```
      }
    }
  }
```

`_score` 값이 높은 순서대로 결과가 나옵니다. title에서 일치한 문서의 점수가 content에서만 일치한 문서보다 높으면 `title^3` 가중치가 동작하고 있는 것입니다.

[CAPTURE NEEDED: Kibana Dev Tools에서 multi_match 검색 실행 - _score 기준 정렬 결과 확인]

8.6 Fuzzy 검색 + Kibana 실습

사용자가 "갈럭시"라고 오타를 입력해도 "갤럭시"를 찾아주는 기능이 Fuzzy 검색 입니다. Elasticsearch는 편집 거리(Edit Distance) 를 기준으로 판단합니다. 한 글자를 바꾸거나, 넣거나, 빼서 원래 단어가 되는지 계산합니다. "갈럭시"에서 "갈"을 "갤"로 바꾸면 "갤럭시"가 되므로 편집 거리는 1입니다. 한글은 완성형 문자이므로 자모 단위와 다를 수 있지만 완성형 글자 단위에서 한 글자 차이는 편집 거리 1로 처리됩니다.

위 검색 코드에서 이미 `.fuzziness("AUTO")` 를 설정해 놓았습니다. AUTO는 단어 길이에 따라 허용하는 편집 거리를 자동으로 조절합니다.

단어 길이	허용 편집 거리	예시
0~2자	0 (오타 불허)	"ab" -> 정확히 일치해야 함
3~5자	1	"갈럭시" -> "갤럭시" 허용
6자 이상	2	"갈럭시노트" -> "갤럭시노트" 허용

Kibana Dev Tools에서 직접 테스트해 봅니다. `localhost:5601` 에 접속합니다.

Kibana 메인 화면 - 왼쪽 메뉴에서 Dev Tools를 찾는다

Dev Tools 진입 - 검색 쿼리를 직접 실행할 수 있는 콘솔이 열린다

Dev Tools 콘솔 - 왼쪽에 쿼리를 작성하고 오른쪽에서 결과를 확인한다

먼저 정상 검색을 실행합니다.

```
GET /devices/_search
{
  "query": {
    "multi_match": {
```

```
      "query": "갤럭시",
      "fields": ["title^3", "content"]
    }
  }
}
```

제목이나 내용에 “갤럭시”가 포함된 문서가 검색됩니다.

이번에는 오타를 넣어서 Fuzzy 검색을 테스트합니다.

```
GET /devices/_search
{
  "query": {
    "multi_match": {
      "query": "갈럭시",
      "fields": ["title^3", "content"],
      "fuzziness": "AUTO"
    }
  }
}
```

“갈럭시”로 검색했지만 “갤럭시”가 포함된 문서가 나옵니다. 편집 거리 1 이내이므로 AUTO가 허용한 것입니다.

Kibana Fuzzy 검색 - “갈럭시”로 검색해도 “갤럭시” 문서가 나온다

Kibana에서 직접 문서를 삽입할 수도 있습니다.

Kibana에서 문서 직접 삽입 - PUT 요청으로 ES에 문서를 넣을 수 있다

한 가지 주의할 점이 있습니다. Kibana에서 ES에 직접 넣은 데이터는 RDB에 들어가지 않습니다. 이중 저장은 Spring Boot 앱을 통해서만 동작합니다. 색인 카드함에 카드만 넣고 책장에는 책을 꽂지 않은 상태입니다.

H2 확인 - Kibana에서 넣은 데이터는 RDB에 없다

Postman에서 오타 검색도 확인합니다.

Postman Fuzzy 검색 - 앱을 통한 오타 검색도 정상 동작한다

중간 확인: Fuzzy 검색이 오타를 허용하는가

“갈럭시”로 검색했을 때 “갤럭시”가 포함된 문서가 반환되면 성공입니다. 정상 검색(“갤럭시”)과 오타 검색(“갈럭시”)의 결과를 비교합니다. 같은 문서가 나오되 `_score` 가 정상 검색보다 낮으면 Fuzzy 매

칭이 동작한 것입니다. “갤럭시” 대신 “갤럭” 처럼 두 글자 이상 차이가 나는 단어로 검색하면 결과가 나오지 않습니다. 편집 거리 제한이 동작하고 있기 때문입니다.

비유	기술 용어	정식 정의
책장 첫 칸부터 끝까지 훑기	LIKE ‘%키워드%’	와일드카드 패턴 매칭. 앞에 %가 붙으면 인덱스를 사용할 수 없어 Full Table Scan이 발생한다
색인 카드함	Elasticsearch	Apache Lucene 기반의 분산 검색 엔진. 역인덱스 구조로 대량 데이터에서 밀리초 단위 전문 검색을 제공한다
단어별 위치 목록 카드	역인덱스 (Inverted Index)	단어를 키로, 해당 단어가 포함된 문서 목록을 값으로 저장하는 자료 구조. 일반 인덱스의 반대 방향
책장에 책 꽂기 + 카드 쓰기	이중 저장 (Dual Write)	하나의 데이터를 RDB와 검색 엔진에 동시에 저장하는 패턴. 저장 일관성 관리가 필요하다
카드에서 위치 확인 후 책장에서 꺼내기	ES 검색 -> RDB 재조회	검색 엔진에서 id를 찾고 RDB에서 원본 엔티티를 조회하는 패턴. 검색 성능과 데이터 정합성을 모두 확보한다
오타를 허용하는 카드함	Fuzzy 검색	편집 거리(삽입, 삭제, 치환) 이내의 유사 단어를 매칭하는 검색 방식. fuzziness AUTO는 단어 길이에 따라 허용 거리를 자동 조절한다

이것만은 기억하자

LIKE 검색은 데이터가 많아지면 테이블 전체를 훑어야 하므로 느려집니다. Elasticsearch는 역인덱스라는 색인 구조를 미리 만들어 놓고 키워드로 문서 위치를 바로 찾습니다. 저장할 때 RDB와 ES에 동시에 넣고 검색할 때 ES에서 찾은 id로 RDB의 원본을 가져오는 것이 기본 패턴입니다. Fuzzy 검색을 쓰면 사용자가 오타를 내도 원래 의도한 결과를 돌려줄 수 있습니다.

다음 장에서는 시스템 간 연동을 자동화해야 하는 상황이 옵니다.

9장. 그 알림, 꼭 직접 전해야 해?

수동 동기화의 한계

설정 파일을 GitHub에서 관리하고 있었습니다. 누군가 README를 수정하면 서버 쪽 파일도 바꿔야 했습니다. 처음에는 수동으로 했습니다. GitHub에서 파일을 열고, 복사하고, 서버에 붙여넣고. 하루에 한두 번이면 참을 만했습니다.

수정 빈도가 늘었습니다. 하루에 열 번 넘게 바뀌는 날이 생겼습니다.

팀장: "README 바뀔 때마다 서버 파일 수동으로 고치고 있어?"

오픈이: "네, 지금은 그렇게 하고 있어요."

팀장: "자동화해. API로 변경 감지하고 바로 반영되게."

(API로 감지까지는 되겠는데, 반영은 어떻게 하지?)

처음 떠오른 방법은 단순했습니다. 변경을 감지한 서버가 반영할 서버의 API를 직접 호출하는 것입니다.

선배: "그러면 반영하는 서버가 내려가 있으면 어떻게 돼?"

오픈이: "요청이 실패하겠지."

선배: "실패한 건 누가 다시 보내? 감지 서버가 재시도 로직까지 갖고 있어야 해. 반영 서버가 두 대로 늘어나면? 셋으로 늘어나면?"

(직접 호출하면 보내는 쪽이 받는 쪽 상태까지 신경 써야 하는구나.)

우체국을 떠올려 봅니다.

편지를 보내는 사람이 받는 사람 집까지 직접 찾아간다고 생각합니다. 받는 사람이 집에 없으면 헛걸음입니다. 받는 사람이 이사하면 새 주소를 알아내야 합니다. 받는 사람이 세 명이면 세 군데를 돌아다녀야 합니다. 보내는 사람이 할 일이 너무 많습니다.

우체국이 있으면 달라집니다. 보내는 사람은 우체국 접수대에 편지를 맡기면 끝입니다. 접수대는 우편번호를 보고 편지를 해당 사서함에 넣어 줍니다. 받는 사람은 자기 사서함을 열어 보면 됩니다. 보내는

사람은 받는 사람이 집에 있는지, 몇 명인지 알 필요가 없습니다. 받는 사람도 보내는 사람이 누구인지 신경 쓸 필요가 없습니다.

RabbitMQ 가 이 우체국입니다.

보내는 사람이 Producer 입니다. 접수대가 Exchange 입니다. 우편번호가 Routing Key 입니다. 사서함이 Queue 입니다. 받는 사람이 Consumer 입니다.

선배: "메시지 큐를 쓰면 보내는 쪽은 큐에 넣기만 하면 돼. 받는 쪽이 죽어 있어도 메시지는 큐에 남아 있고, 살아나면 그때 가져가."

(우체국이 중간에 편지를 보관해 주는 거구나. 받는 사람이 부재중이어도 편지가 사라지지 않는 거지.)

오픈이: "만들어 볼게요."

이 장의 실습 코드는 아래 레포에서 확인할 수 있습니다.

```
git clone https://github.com/metacoding-11-spring-reference/rabbitmq-docker
git clone https://github.com/metacoding-11-spring-reference/rabbitmq-producer
git clone https://github.com/metacoding-11-spring-reference/rabbitmq-consumer
```

```
rabbitmq-docker/
└── docker-compose.yml          [실습] RabbitMQ 구성

rabbitmq-producer/
├── GitHubClient.java           [설명] GitHub API 호출 (sha 조회 + README 내용)
├── PollingScheduler.java       [설명] @Scheduled 폴링 + 변경 감지
├── RabbitProducer.java         [실습] RabbitTemplate 메시지 발행
├── RabbitDTO.java              [설명] 메시지 DTO (repo, sha, content, timestamp)
├── RabbitConfig.java           [실습] Exchange/Queue/Binding 설정
└── application.properties      [실습] RabbitMQ + GitHub 접속 정보

rabbitmq-consumer/
├── RabbitConsumer.java         [실습] @RabbitListener 수신 + 파일 반영
├── RabbitDTO.java              [설명] 메시지 DTO (Producer와 동일)
├── RabbitConfig.java           [실습] Queue 빈 + JSON 컨버터
└── application.properties      [실습] RabbitMQ 접속 정보
```

기술 파트

이번 챕터의 실습 흐름

9.1 시나리오와 전체 구조

전체 흐름을 정리합니다.

GitHub에 README가 수정되면 Producer가 주기적으로 폴링하여 변경을 감지합니다. 변경이 감지되면 Producer는 메시지를 RabbitMQ의 Exchange에 발행합니다. Exchange는 Routing Key를 보고 해당 Queue에 메시지를 넣습니다. Consumer는 Queue를 구독하고 있다가 메시지가 도착하면 로컬 파일에 반영합니다.

전체 흐름 - GitHub 변경이 Producer를 거쳐 RabbitMQ를 통해 Consumer에 전달된다

핵심 개념 세 가지를 짚고 넘어갑니다.

Exchange 는 메시지를 받아서 Queue로 전달하는 분류 지점입니다. 우체국 접수대에 해당합니다. 이 실습에서는 Direct 방식을 사용합니다. Direct Exchange는 메시지의 Routing Key가 정확히 일치하는 Queue에만 메시지를 전달합니다.

Queue 는 메시지를 보관하는 대기열입니다. 우체국 사서함에 해당합니다. Consumer가 꺼내갈 때까지 메시지가 남아 있습니다.

Routing Key 는 메시지에 붙는 라벨입니다. 우편번호에 해당합니다. Exchange가 이 값을 보고 어떤 Queue로 보낼지 결정합니다. 이 실습에서는 `readme.changed` 라는 Routing Key를 사용합니다.

Exchange와 Queue를 연결하는 규칙을 Binding 이라고 합니다. “이 우편번호의 편지는 이 사서함에 넣어라”는 규칙을 등록하는 것입니다.

9.2 실습 환경

RabbitMQ를 Docker로 실행합니다. 아래 코드를 `docker-compose.yml` 에 작성합니다.

```yaml
services:
  rabbitmq:
    image: rabbitmq:3-management
    ports:
      - "5672:5672"
      - "15672:15672"
```

5672는 RabbitMQ의 메시지 통신 포트이고 15672는 관리 콘솔 웹 포트입니다. `rabbitmq:3-management` 이미지는 관리 콘솔이 포함된 버전입니다.

```bash
docker-compose up -d
```

컨테이너가 올라오면 RabbitMQ 상태를 확인합니다. Management UI가 준비되기까지 10~20초 정도 걸릴 수 있습니다.

브라우저에서 `http://localhost:15672` 에 접속합니다. 로그인 화면이 보이면 성공입니다. 기본 계정은 `guest / guest` 입니다.

RabbitMQ 관리 콘솔 로그인 화면 - guest/guest로 접속한다

로그인하면 관리 콘솔 메인 화면이 나옵니다.

RabbitMQ 관리 콘솔 - 연결, 채널, Exchange, Queue 상태를 한눈에 볼 수 있다

상단 탭을 살펴봅니다. Overview 는 전체 요약, Connections 는 현재 접속 중인 클라이언트 목록, Channels 는 메시지 통신 채널, Exchanges 는 메시지를 분류하는 접수대 목록, Queues 는 메시지가 대기하는 사서함 목록입니다. 실습 중에는 주로 Exchanges 탭과 Queues 탭을 확인합니다. 아직 Producer를 실행하지 않았으므로 Exchanges 탭에 기본 Exchange만 보이고 Queues 탭은 비어 있습니다.

GitHub Personal Access Token 발급

Producer가 GitHub API를 호출하려면 인증 토큰이 필요합니다. GitHub에서 Personal Access Token을 발급합니다.

1. GitHub에 로그인합니다.
2. 오른쪽 상단 프로필 아이콘을 클릭하고 Settings 를 선택합니다.
3. 왼쪽 사이드바 맨 아래 Developer settings 를 클릭합니다.
4. Personal access tokens > Tokens (classic) 을 선택합니다.
5. Generate new token > Generate new token (classic) 을 클릭합니다.
6. Note에 `rabbitmq-demo` 등 용도를 적고, Expiration은 실습 기간에 맞춰 설정합니다.
7. Scope는 `repo` 에 체크합니다. 공개 레포만 사용한다면 체크 없이도 됩니다.
8. Generate token 을 클릭하고 표시된 토큰을 복사합니다.

토큰은 이 화면을 벗어나면 다시 볼 수 없으므로 바로 복사해 둡니다. 이 토큰은 Producer의 `application.properties` 에서 사용합니다.

[CAPTURE NEEDED: GitHub Developer settings > Personal access tokens 화면 - Generate new token 버튼 위치]

GitHub에 테스트용 레포를 하나 만들어 둡니다. 레포 이름은 `config-readme` 로 하고 README.md 파일을 생성합니다.

GitHub에 config-readme 레포 생성 - 이 레포의 README 변경을 감지할 것이다

9.3 Producer 구현

Producer는 두 가지 일을 합니다. GitHub의 README 변경을 감지하는 것과 변경이 감지되면 RabbitMQ에 메시지를 발행하는 것입니다.

먼저 RabbitMQ 연결 정보와 GitHub 설정을 `application.properties` 에 작성합니다.

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
rabbit.exchange=github.events
rabbit.queue=repo-updates
rabbit.routing-key=readme.changed
github.owner=자신의깃헙계정
github.repo=config-readme
github.readme-path=README.md
github.poll-interval-ms=60000
```

`github.poll-interval-ms=60000` 은 60초마다 GitHub API를 호출하겠다는 뜻입니다.

Exchange, Queue, Binding을 Spring Bean으로 등록합니다. 아래 코드를 `RabbitConfig.java` 에 작성합니다.

```java
@Bean
public DirectExchange exchange() {
    return new DirectExchange(exchangeName);
}
@Bean
public Queue queue() {
    return new Queue(queueName, true);
}
@Bean
public Binding binding(Queue queue, DirectExchange exchange) {
    return BindingBuilder.bind(queue).to(exchange).with(routingKey);
}
```

`new Queue(queueName, true)` 의 두 번째 인자 `true` 는 durable 설정입니다. RabbitMQ가 재시작되어도 Queue가 유지됩니다.

`BindingBuilder.bind(queue).to(exchange).with(routingKey)` 는 "이 Queue를 이 Exchange에 연결하되, 이 Routing Key의 메시지만 받아라"는 규칙을 등록합니다.

메시지 발행 코드를 봅니다. `RabbitProducer.java` 의 핵심은 한 줄입니다.

```
public void send(RabbitDTO message) {
    rabbitTemplate.convertAndSend(exchange, routingKey, message);
}
```

`convertAndSend` 는 객체를 JSON으로 변환하여 지정한 Exchange에 Routing Key와 함께 발행합니다. 우체국 접수대에 "이 우편번호로 보내 주세요"라고 맡기는 것과 같습니다.

변경 감지는 `PollingScheduler.java` 가 담당합니다.

```
private String lastSha = null;

@Scheduled(fixedRateString = "${github.poll-interval-ms}")
public void checkForReadmeChange() {
    String latestSha = gitHubClient.fetchLatestSha();
    if (latestSha.equals(lastSha)) { return; }
    lastSha = latestSha;
    String content = gitHubClient.fetchReadmeContent();
    RabbitDTO message = RabbitDTO.builder()
            .repo(gitHubClient.getRepoFullName()).sha(latestSha)
            .content(content).timestamp(LocalDateTime.now()).build();
    rabbitProducer.send(message);
}
```

`@Scheduled` 로 60초마다 GitHub API를 호출합니다. 마지막으로 확인한 커밋의 SHA 값과 현재 SHA 값을 비교해서 다르면 변경이 있다고 판단합니다. 변경이 있으면 README 내용을 가져와서 RabbitMQ에 메시지를 발행합니다.

`GitHubClient.java` 는 GitHub REST API를 호출하여 최신 커밋의 SHA를 조회합니다.

```
public String fetchLatestSha() {
    String url = String.format(
        "https://api.github.com/repos/%s/%s/commits?path=%s&per_page=1",
        owner, repo, readmePath);
    ResponseEntity<List<Map<String, Object>>> response =
        restTemplate.exchange(url, HttpMethod.GET, ...);
    Object sha = commitObj.get("sha");
    return sha != null ? sha.toString() : null;
}
```

`per_page=1` 로 가장 최신 커밋 하나만 가져옵니다. 메시지에 담기는 DTO는 단순합니다.

```
public class RabbitDTO {
    private String repo;
    private String sha;
    private String content;
    private LocalDateTime timestamp;
}
```

Producer를 실행합니다.

Producer 서버 실행 - 60초마다 GitHub을 폴링한다

GitHub에서 README를 수정하고 커밋합니다.

GitHub에서 README 수정

Producer 로그에서 변경 감지를 확인합니다.

Producer가 SHA 변경을 감지하고 메시지를 발행한 로그

중간 확인: 메시지가 Queue에 들어갔는가

RabbitMQ 관리 콘솔에서 Queues 탭을 클릭합니다. `repo-updates` Queue가 목록에 보이고 Ready 컬럼에 숫자가 1 이상이면 메시지가 정상적으로 큐잉된 것입니다. Consumer를 아직 실행하지 않았으므로 메시지가 Queue에 대기하고 있어야 합니다. Ready가 0이면 Producer 로그에서 에러가 없었는지 확인합니다.

[CAPTURE NEEDED: RabbitMQ Queues 탭 - repo-updates Queue의 Ready 메시지 수 확인]

9.4 Consumer 구현

Consumer는 Queue에서 메시지를 꺼내 로컬 파일에 반영합니다. `RabbitConfig.java` 에 Queue와 JSON 컨버터를 등록합니다.

```
@Bean
public Queue queue() {
    return new Queue(queueName, true);
}
@Bean
public MessageConverter jsonMessageConverter() {
    return new Jackson2JsonMessageConverter();
}
```

`Jackson2JsonMessageConverter` 를 Bean으로 등록하면 RabbitMQ에서 받은 JSON 메시지가 자동으로 자바 객체로 변환됩니다.

`RabbitConsumer.java` 가 메시지를 수신합니다.

```
@RabbitListener(queues = "${rabbit.queue}")
public void receive(RabbitDTO message) {
    System.out.println("=== 메시지 수신 ===");
    System.out.println("Repository: " + message.getRepo());
    System.out.println("SHA: " + message.getSha());
    patchFile(message.getContent());
}
```

`@RabbitListener` 는 지정한 Queue를 구독합니다. 메시지가 도착하면 `receive` 메서드가 자동으로 호출됩니다. 우체국 사서함 앞에서 편지가 오기를 기다리고 있는 것과 같습니다.

`patchFile` 은 파일 내용을 비교하여 변경이 있을 때만 덮어씁니다.

```
private void patchFile(String newContent) throws IOException {
    File file = new File(README_PATH);
    String oldContent = Files.readString(file.toPath(), StandardCharsets.UTF_8);
    if (Objects.equals(oldContent.trim(), newContent.trim())) { return; }
    Files.copy(file.toPath(), backupFile.toPath());
    writeContent(file, newContent);
}
```

기존 파일을 백업한 뒤 새 내용으로 덮어씁니다. 내용이 같으면 아무것도 하지 않습니다.

Consumer의 `application.properties` 는 간단합니다.

```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
rabbit.queue=repo-updates
```

Consumer는 Queue 이름만 알면 됩니다. Exchange나 Routing Key는 Producer 쪽에서 설정합니다. 받는 사람은 자기 사서함 번호만 알면 되는 것과 같습니다.

9.5 통합 테스트 시나리오

세 서버를 모두 실행합니다. Docker(RabbitMQ), Producer, Consumer 순서입니다.

Docker, Producer, Consumer 세 서버가 모두 실행 중이다

RabbitMQ 관리 콘솔에서 Exchange 탭을 확인합니다.

Exchanges 탭 - Producer가 등록한 github.events Exchange가 보인다

Exchange 상세에서 Binding 정보를 확인합니다.

Exchange 상세 - repo-updates Queue에 readme.changed Routing Key로 바인딩되어 있다

Queue 상세에서 메시지 수와 상태를 확인합니다.

Queue 상세 - repo-updates Queue의 메시지 수와 소비 상태가 보인다

Binding 정보도 확인합니다. Exchange에서 Queue로의 연결 규칙입니다.

Binding 확인 - readme.changed Routing Key로 바인딩되어 있다

이제 GitHub에서 README를 수정하고 커밋합니다.

GitHub에서 README를 수정하고 커밋한다

Producer가 변경을 감지합니다.

Producer가 SHA 변경을 감지하고 메시지를 발행한다

Consumer가 메시지를 수신합니다.

Consumer가 메시지를 수신하고 파일 반영을 시작한다

로컬 README 파일이 자동으로 업데이트됩니다.

로컬 README 파일이 GitHub의 내용으로 자동 업데이트되었다

백업 파일도 생성되었는지 확인합니다.

이전 내용이 백업 파일로 저장되어 있다

관리 콘솔에서 Queue에 들어온 메시지 내용을 직접 확인할 수도 있습니다. Queue 상세 화면에서 Get Messages를 클릭합니다.

Queue 메시지 payload - repo, sha, content, timestamp가 JSON으로 들어 있다

GitHub README 수정부터 로컬 파일 반영까지 사람이 개입하지 않았습니다. 편지를 우체국에 맡기면 알아서 사서함에 도착하듯이 메시지가 자동으로 흘러갔습니다.

중간 확인: Consumer가 메시지를 소비했는가

RabbitMQ 관리 콘솔의 Queues 탭에서 `repo-updates` Queue를 확인합니다. Consumer 실행 전에 Ready 에 쌓여 있던 메시지 수가 Consumer 실행 후 0으로 줄어들면 성공입니다. Consumer 터미널에 “메시지 수신” 로그가 출력되고 로컬 README 파일의 내용이 GitHub에서 수정한 내용과 같으면 전체 흐름이 정상 동작한 것입니다.

전체 흐름을 다시 한번 확인합니다. GitHub에서 README를 한 번 더 수정하고 커밋합니다. 60초 이내에 Producer 로그에서 SHA 변경 감지가 출력되고, Consumer 로그에서 메시지 수신이 출력되고, 로컬 README 파일이 업데이트되면 end-to-end 동작이 검증된 것입니다.

9.6 실무 확장 포인트

이 실습은 가장 단순한 형태입니다. 실무에서는 몇 가지를 더 고려합니다.

Dead Letter Queue 는 처리에 실패한 메시지를 모아 두는 별도의 Queue입니다. Consumer가 메시지를 처리하다가 예외가 발생하면 메시지가 사라지지 않고 Dead Letter Queue로 이동합니다. 반송 편지함이라고 생각하면 됩니다. 나중에 실패 원인을 분석하거나 재처리할 수 있습니다.

Webhook vs Polling 도 고려 대상입니다. 이 실습에서는 60초마다 GitHub API를 호출하는 Polling 방식을 사용했습니다. GitHub Webhook을 사용하면 README가 변경되는 즉시 GitHub이 Producer에게 알려 줍니다. 우체국에 직접 가서 편지가 왔는지 확인하는 것과 우체부가 집 앞까지 와서 알려 주는 것의 차이입니다. Webhook은 실시간성이 좋지만 외부에서 접근할 수 있는 URL이 필요합니다.

다중 Consumer 도 가능합니다. 같은 Queue를 여러 Consumer가 구독하면 메시지가 라운드 로빈 방식으로 분배됩니다. 하나의 사서함을 여러 사람이 번갈아 확인하는 것과 같습니다. 처리량이 부족할 때 Consumer만 늘리면 됩니다. Producer는 변경할 필요가 없습니다.

비유	기술 용어	정식 정의
편지 보내는 사람	Producer	메시지를 생성하여 Exchange에 발행하는 주체. RabbitTemplate의 convertAndSend 메서드를 사용한다
우체국 접수대	Exchange	메시지를 받아 Routing Key를 기준으로 적절한 Queue에 라우팅하는 분류 지점. Direct, Topic, Fanout 등의 타입이 있다
우편번호	Routing Key	메시지에 붙는 라벨. Exchange가 이 값을 보고 어떤 Queue로 메시지를 전달할지 결정한다
사서함	Queue	메시지를 보관하는 대기열. Consumer가 꺼내갈 때까지 메시지가 남아 있으며 durable 설정 시 서버 재시작에도 유지된다
편지 받는 사람	Consumer	Queue를 구독하여 메시지를 수신하는 주체. @RabbitListener로 특정 Queue를 구독한다
사서함과 접수대를 연결하는 규칙	Binding	Exchange와 Queue를 연결하는 규칙. 특정 Routing Key의 메시지를 특정 Queue로 전달하도록 설정한다
반송 편지함	Dead Letter Queue	처리에 실패한 메시지를 모아 두는 별도의 Queue. 실패 원인 분석이나 재처리에 사용된다

이것만은 기억하자

시스템 간에 데이터를 전달할 때 직접 호출하면 받는 쪽의 상태에 보내는 쪽이 영향을 받습니다. RabbitMQ를 사이에 두면 보내는 쪽은 메시지를 맡기기만 하고 받는 쪽은 자기 속도에 맞춰 가져갑니다. Exchange가 Routing Key를 보고 메시지를 적절한 Queue에 넣어 주고 Consumer는 Queue를 구독하여 메시지가 도착하면 처리합니다. 받는 쪽이 잠시 내려가 있어도 메시지는 Queue에 남아 있습니다.

Docker 명령어 앞에서 얼어붙던 터미널이 떠오릅니다. 1장에서 처음 컨테이너를 띄우고 소셜 로그인을 붙이고 세션을 공유하고 파일을 올리고 실시간 통신을 구현하고 검색 엔진을 연결하고 메시지 큐까지 왔습니다. 어느 기술도 처음부터 쉽지 않았지만 한 번 만들어 보고 나면 두 번째는 덜 무서웠습니다. 에필로그에서 이 여정을 돌아봅니다.

맺음말

아홉 장의 티켓이 끝났습니다.

Docker 앞에서 멈췄던 터미널에 이제 `docker compose up` 을 치는 데 망설임이 없을 겁니다. OAuth 시퀀스 다이어그램의 화살표가 열다섯 개쯤 되어도, 결국 토큰을 주고받는 흐름이라는 걸 압니다. Redis가 왜 필요한지, S3에 파일을 어떻게 올리는지, 실시간 통신을 어떤 기준으로 고르는지. 한 번씩은 직접 돌려 봤습니다.

물론 이 책에서 다룬 건 각 기술의 전체 그림 중 입구에 해당하는 부분입니다. Redis의 클러스터 구성, Elasticsearch의 분석기 튜닝, RabbitMQ의 장애 복구 전략처럼 깊이 들어가야 할 영역은 아직 남아 있습니다. 그건 다음 단계입니다.

다만 한 가지는 달라졌을 겁니다. 처음 보는 기술 이름 앞에서 얼어붙는 시간이 줄었습니다. “일단 로컬에서 띄워 보자”가 자연스러운 첫 반응이 되었다면, 이 책은 제 역할을 한 셈입니다.

다음 티켓은 사수가 아니라 여러분이 직접 고르는 겁니다.